
Big Data, ML e IA

Domina el big data, machine learning e inteligencia artificial con proyectos prácticos

Alex Goyzueta Delgado

alexgoyzueta2018@gmail.com

Big Data, ML e IA

Domina el big data, machine learning e inteligencia artificial con proyectos prácticos

Autor: Alex Goyzueta Delgado

Colección: Data & Analytics Mastery

Libro 5 de 5

Edición 1.0 — Julio 2026

© 2026 Alex Goyzueta Delgado. Todos los derechos reservados.

Ninguna parte de esta publicación puede ser reproducida, distribuida o transmitida en forma alguna ni por ningún medio, incluyendo fotocopiado, grabación u otros métodos electrónicos o mecánicos, sin el permiso previo por escrito del autor, excepto en el caso de citas breves incluidas en reseñas críticas y otros usos no comerciales permitidos por la ley de derechos de autor.

Contacto: alex.goyzueta@email.com

Tecnologías cubiertas:

- Apache Spark / PySpark
- Scikit-learn
- TensorFlow / Keras

- MLflow
- Google Vertex AI / BigQuery ML
- LangChain / LLMs
- Streamlit

Datos utilizados: Dataset TechStore (creado específicamente para esta colección)

Este libro es el resultado de años de experiencia práctica en ciencia de datos, machine learning e ingeniería de datos. Su objetivo es compartir conocimiento práctico y aplicable que ayude a profesionales y estudiantes a dominar el ecosistema completo de datos y AI.

Dedicatoria

A todos los que creen que los datos cuentan historias que merecen ser escuchadas.

A quienes no temen preguntar "¿y si...?" y se lanzan a descubrir la respuesta.

A los curiosos, los persistentes, los que saben que cada modelo es una aproximación, cada predicción una hipótesis, y cada insight una oportunidad.

Esta colección completa es para ustedes.

Prefacio

Has llegado al final del camino... o mejor dicho, al comienzo de una nueva etapa.

La colección *Data & Analytics Mastery* te ha acompañado desde los fundamentos del análisis de datos (Libro 1), pasando por Python y la nube (Libro 2), el Business Intelligence y Power BI (Libro 3), y la ingeniería de datos moderna con dbt, Airflow y Docker (Libro 4). En cada etapa, el dataset TechStore ha sido nuestro hilo conductor, evolucionando y creciendo en complejidad.

Este quinto y último libro es la culminación de ese viaje. Aquí no solo aplicarás big data y machine learning, sino que integrarás todo lo aprendido para construir sistemas inteligentes que transforman datos en decisiones autónomas, predicciones y experiencias personalizadas.

¿Por qué este libro?

El mundo de los datos ha cambiado drásticamente. El big data dejó de ser una novedad para convertirse en el estándar. El machine learning pasó de ser una especialidad a una competencia central. Y la inteligencia artificial generativa ha democratizado el acceso a capacidades que hasta hace poco parecían ciencia ficción.

Este libro tiende un puente entre estos mundos. No solo te enseña PySpark o scikit-learn de forma aislada; te muestra cómo combinar big data, ML, MLOps e IA generativa en un flujo de trabajo cohesivo y profesional.

Estructura del libro

El libro está organizado en cuatro proyectos progresivos:

Proyecto 1 — Fundamentos de Big Data: Comenzamos con los conceptos fundamentales del big data y PySpark. Aprenderás a procesar volúmenes masivos de datos, trabajar con Spark SQL, manejar datos en streaming y construir pipelines de datos a escala con el dataset TechStore.

Proyecto 2 — Machine Learning: Entramos de lleno en el machine learning con scikit-learn. Desde la regresión lineal hasta random forests y XGBoost, pasando por clustering y sistemas de recomendación. Cada algoritmo se aplica a problemas reales de TechStore.

Proyecto 3 — Deep Learning, NLP y MLOps: Exploramos redes neuronales con TensorFlow, procesamiento de lenguaje natural aplicado a reseñas de clientes, forecasting de ventas con series temporales, y MLOps con MLflow para llevar modelos a producción.

Proyecto 4 — Inteligencia Artificial y Despliegue: El gran final. Trabajamos con LLMs y RAG usando LangChain, machine learning en la nube con Vertex AI y BigQuery ML, y culminamos con un proyecto final que integra toda la plataforma.

¿Qué necesitas saber?

Este libro asume que tienes conocimientos sólidos de Python, SQL y fundamentos de datos (cubiertos en los libros anteriores de la colección). Si has seguido la serie desde el principio, estás perfectamente preparado.

El dataset TechStore

Una vez más, trabajaremos con TechStore, nuestro dataset de comercio electrónico que ha evolucionado a lo largo de la colección. En este libro, TechStore alcanza su máxima expresión: millones de transacciones simuladas, reseñas de clientes, datos de navegación en tiempo real y un catálogo completo de productos. Es el campo de prueba perfecto para técnicas de big data y machine learning.

Comencemos

El viaje de cinco libros termina aquí, pero tu dominio del ecosistema de datos comienza ahora. Abre tu terminal, prepara tu entorno Python, y vamos a construir el futuro con datos.

— Alex Goyzueta Delgado

Introducción

Bienvenido a *Big Data, ML e IA*, el quinto y último libro de la colección *Data & Analytics Mastery*.

De dónde venimos

En el Libro 1 aprendiste los fundamentos del análisis de datos con Python, SQL y estadística. En el Libro 2 expandiste tus habilidades con la nube y BigQuery. En el Libro 3 dominaste el Business Intelligence y Power BI. En el Libro 4 te convertiste en un data engineer capaz de construir pipelines completos con dbt, Airflow y Docker.

Ahora es momento de culminar ese viaje con el libro más ambicioso de la colección.

Hacia dónde vamos

Este libro cubre cuatro grandes áreas que definen el estado del arte en datos:

Big Data

Cuando los datos superan la capacidad de procesamiento de una sola máquina, entran en juego herramientas como Apache Spark. Aprenderás PySpark para procesar datos a escala, trabajar con DataFrames distribuidos, optimizar consultas y manejar datos en streaming.

Machine Learning

El corazón del libro. Construirás modelos predictivos completos usando scikit-learn: regresión, clasificación, clustering, sistemas de recomendación y más. Cada modelo se aplica a problemas de negocio reales de TechStore.

Deep Learning y MLOps

Las redes neuronales profundas, el procesamiento de lenguaje natural y el forecasting con series temporales llevan tus capacidades al siguiente nivel. Y con MLOps aprenderás a gestionar el ciclo de vida completo de los modelos: experimentación, versionado, despliegue y monitoreo.

Inteligencia Artificial Generativa

El libro te introduce en el mundo de los LLMs, RAG (Retrieval-Augmented Generation) y LangChain. Aprenderás a construir asistentes inteligentes que responden preguntas sobre tus datos usando modelos de lenguaje avanzados.

Enfoque práctico

Cada capítulo incluye ejemplos de código completos y ejercicios. Los proyectos integradores (checkpoints) te retan a combinar lo aprendido en escenarios del mundo real. El proyecto final del

libro integra todas las técnicas en una plataforma de datos con IA.

Convenciones usadas en el libro

- **Código:** Los bloques de código en Python se muestran con fondo oscuro.
- **Comandos:** Los comandos de terminal se indican con el prefijo ``$``.
- **Ejercicios:** Al final de cada capítulo hay ejercicios para practicar.
- **Checkpoints:** Los proyectos integradores combinan múltiples capítulos.
- **Notas:** Las notas importantes se indican con el ícono .

Requisitos técnicos

- Python 3.10+
- PySpark
- Scikit-learn
- TensorFlow
- MLflow
- Google Cloud SDK (para la parte cloud)
- LangChain
- Jupyter Notebook o VS Code

Todas las instrucciones de instalación están en el Apéndice C.

El dataset TechStore en este libro

TechStore ha evolucionado. Para este libro, el dataset incluye:

- ``techstore_transactions``: 10 millones de transacciones simuladas
- ``techstore_customers``: 100,000 perfiles de clientes
- ``techstore_products``: 5,000 productos con categorías
- ``techstore_reviews``: 500,000 reseñas de productos
- ``techstore_events``: 50 millones de eventos de navegación
- ``techstore_inventory``: Datos de inventario en tiempo real

Los datos están diseñados para exhibir características de big data: volumen, velocidad y variedad.

Prepárate. Este libro te llevará al límite de tus capacidades... y más allá.

Capítulo 1: Introducción al Big Data

1.1 ¿Qué es Big Data?

Big data no es solo "datos grandes". Es un fenómeno definido por las **5 V**:

Volumen	Cantidad masiva de datos	10M transacciones, 50M eventos
Variedad	Diferentes formatos y fuentes	CSV, JSON, Parquet, texto libre
Valor	Capacidad de extraer insights	Predicciones, segmentaciones, recomendaciones

1.1.1 El ecosistema Hadoop

Hadoop fue el pionero que democratizó el big data:

```
HDFS (almacenamiento distribuido)
├── YARN (gestión de recursos)
│   └── MapReduce (procesamiento)
│       └── Ecosistema: Hive, Pig, HBase, Spark
```

HDFS divide archivos en bloques (128 MB por defecto) y los replica (3 copias) en múltiples nodos. Esto proporciona tolerancia a fallos y paralelismo.

MapReduce procesa datos en dos fases:

- **Map**: Transforma datos en pares clave-valor
- **Reduce**: Agrupa y agrega resultados

1.1.2 Apache Spark: el sucesor

Spark supera a MapReduce al mantener datos en memoria, logrando velocidades 10-100x superiores.

```
# MapReduce mentalidad
datos = leer_desde_disco()
resultado = map_filter(datos) # escribe a disco
final = reduce_agrupa(resultado) # lee de disco

# Spark mentalidad
datos = spark.read.parquet("techstore/")
resultado = datos.filter(...).groupBy(...).agg(...) # todo en memoria
```

Componentes de Spark:

```
graph TD
  A[Spark Core] --> B[Spark SQL]
  A --> C[MLlib]
  A --> D[Structured Streaming]
  A --> E[GraphX]
```

1.2 Arquitectura de Spark

1.2.1 Cluster Mode

```
# Modos de despliegue
modos = {
  "local": "Tu máquina, para desarrollo y pruebas",
  "standalone": "Cluster Spark gestionado por Spark",
  "yarn": "Recursos gestionados por YARN (Hadoop)",
  "kubernetes": "Contenedores orquestados por K8s"
}
```

Componentes del cluster:

- **Driver:** Programa principal con `SparkContext`
- **Master:** Gestiona recursos y planifica tareas
- **Worker:** Ejecuta tareas en sus cores
- **Executor:** Proceso JVM en cada worker que ejecuta tareas

1.2.2 SparkSession

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("TechStore-Analytics") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.sql.adaptive.coalescePartitions.enabled", "true") \
    .config("spark.driver.memory", "4g") \
    .getOrCreate()

print(f"Spark version: {spark.version}")
# Spark version: 3.5.0
```

1.2.3 Lazy Evaluation

Spark usa evaluación perezosa: las transformaciones se registran en un DAG (Directed Acyclic Graph) y solo se ejecutan cuando una acción las dispara.

```
# Transformaciones (no ejecutan nada)
df = spark.read.parquet("techstore/")
filtered = df.filter(df.amount > 100)
grouped = filtered.groupBy("category").count()

# Acción (dispara la ejecución)
```

```
result = grouped.collect() # aquí Spark ejecuta todo el DAG
```

1.3 TechStore a Escala

Para este libro, generamos datos a escala de big data:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import rand, when, col, expr
import random

spark = SparkSession.builder.appName("TechStore-Gen").getOrCreate()

# Generar 10 millones de transacciones
transactions = spark.range(10_000_000) \
    .withColumn("customer_id", (rand() * 100000).cast("int")) \
    .withColumn("product_id", (rand() * 5000).cast("int")) \
    .withColumn("amount", (rand() * 500 + 5).cast("decimal(10,2)")) \
    .withColumn("quantity", (rand() * 5 + 1).cast("int")) \
    .withColumn("timestamp", expr(
        "timestamp('2025-01-01') + interval rand() * 365 days"
    ))

transactions.write.mode("overwrite") \
    .parquet("data/techstore_transactions/")

print(f"Transacciones generadas: {transactions.count():,}")
# Transacciones generadas: 10,000,000
```

1.4 Comparativa de Rendimiento

```
import time

# Modo clásico (pandas)
import pandas as pd
t0 = time.time()
pdf = pd.read_csv("techstore_10m.csv") # ni siquiera cabe en RAM
t1 = time.time()

# Modo Spark
t2 = time.time()
sdf = spark.read.parquet("data/techstore_transactions/") \
    .filter(col("amount") > 100) \
    .groupBy("product_id") \
    .agg({"amount": "sum"})
sdf.collect()
t3 = time.time()

print(f"Spark: {t3-t2:.2f}s vs Pandas: {t1-t0:.2f}s (si cupiera)")
```

1.5 Ejercicios

1. **Conceptos:** Explica las 5 V del big data con ejemplos de TechStore.
2. **Arquitectura:** Dibuja el DAG de Spark para ``df.filter().groupBy().agg().orderBy().show()``.
3. **SparkSession:** Crea una SparkSession con 2 GB de driver memory y nombre "MiApp".
4. **Generación:** Genera 1 millón de eventos de navegación TechStore con columnas: `user_id`, `page`, `timestamp`, `duration_ms`.
5. **Comparativa:** ¿Qué ventajas tiene Spark sobre Pandas para datasets > 10 GB?

Capítulo 2: PySpark: Procesamiento Distribuido

2.1 RDDs: La Base de Spark

Los RDDs (Resilient Distributed Datasets) son la abstracción fundamental de Spark. Son colecciones inmutables y distribuidas de objetos que pueden procesarse en paralelo.

2.1.1 Creación de RDDs

```
# Desde una colección local
rdd = spark.sparkContext.parallelize([1, 2, 3, 4, 5])
print(rdd.getNumPartitions()) # 4 (por defecto)

# Desde archivo
rdd_text = spark.sparkContext.textFile("data/reviews.txt")
print(rdd_text.count()) # número de líneas

# Con particiones específicas
rdd_opt = spark.sparkContext.parallelize(datos, minPartitions=8)
```

2.1.2 Transformaciones en RDD

```
rdd = spark.sparkContext.parallelize([
    ("electronics", 250.0),
    ("clothing", 45.0),
    ("electronics", 180.0),
    ("food", 15.0),
    ("electronics", 300.0)
])

# map: transforma cada elemento
rdd_map = rdd.map(lambda x: (x[0], x[1] * 1.21)) # +21% IVA

# filter: selecciona elementos
rdd_filter = rdd.filter(lambda x: x[1] > 100)

# flatMap: un elemento puede producir 0, 1 o múltiples
text_rdd = spark.sparkContext.parallelize(["hola mundo", "hola spark"])
words = text_rdd.flatMap(lambda line: line.split(" "))
# ["hola", "mundo", "hola", "spark"]
```

```
# reduceByKey: agrupa por clave y reduce
totales = rdd.reduceByKey(lambda a, b: a + b)
print(totales.collect())
# [("electronics", 730.0), ("clothing", 45.0), ("food", 15.0)]
```

2.2 DataFrames: La API Moderna

Los DataFrames son la evolución de los RDDs. Ofrecen optimización nativa mediante Catalyst Optimizer.

2.2.1 Creación y Lectura

```
# Desde archivo Parquet (recomendado)
df = spark.read.parquet("data/techstore_transactions/")
df.printSchema()

# Desde CSV
df_csv = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("data/techstore_products.csv")

# Desde JSON
df_json = spark.read.json("data/events/")

# Desde una tabla
df_table = spark.sql("SELECT * FROM techstore.transactions")
```

2.2.2 Transformaciones Principales

```
from pyspark.sql.functions import col, sum, avg, count, when, datediff, to_date
from pyspark.sql.window import Window
import pyspark.sql.functions as F

# Selección y filtrado
ventas = df.select("customer_id", "amount", "timestamp") \
    .filter(col("amount") > 0) \
    .filter(col("timestamp").isNotNull())

# Agregaciones
por_categoria = df.groupBy("category") \
    .agg(
        sum("amount").alias("total_ventas"),
        avg("amount").alias("ticket_promedio"),
        count("transaction_id").alias("num_transacciones")
    ) \
    .orderBy(col("total_ventas").desc())

# Columnas condicionales
df_con_segmento = df.withColumn(
    "segmento",
```

```

        when(col("amount") > 500, "Premium")
        .when(col("amount") > 100, "Regular")
        .otherwise("Básico")
    )

    # Window functions
    ventana_cliente = Window.partitionBy("customer_id") \
        .orderBy("timestamp") \
        .rowsBetween(Window.unboundedPreceding, Window.currentRow)

    df_con_acumulado = df.withColumn(
        "gasto_acumulado",
        sum("amount").over(ventana_cliente)
    )

```

2.2.3 SQL Directo

```

# Registrar DataFrame como vista temporal
df.createOrReplaceTempView("ventas")

# Consultas SQL nativas
resultado = spark.sql("""
    SELECT
        category,
        DATE_TRUNC('month', timestamp) as mes,
        ROUND(SUM(amount), 2) as ingresos,
        COUNT(DISTINCT customer_id) as clientes_unicos,
        ROUND(AVG(amount), 2) as ticket_promedio
    FROM ventas
    WHERE amount > 0
    GROUP BY category, DATE_TRUNC('month', timestamp)
    HAVING ingresos > 1000
    ORDER BY mes, ingresos DESC
""")
resultado.show(5)

```

2.3 Optimización de Consultas

2.3.1 Catalyst Optimizer

Spark optimiza automáticamente tu código:

```

# Esto...
resultado = df.filter(col("amount") > 100) \
    .select("customer_id", "amount") \
    .groupBy("customer_id") \
    .agg(sum("amount"))

# ...Catalyst lo optimiza a:
# 1. Predicate pushdown: filtra amount > 100 antes de leer
# 2. Project pruning: solo lee las columnas necesarias
# 3. Optimización de agregación: combine filters and aggregates

```

2.3.2 Particionamiento

```
# Ver particiones actuales
print(f"Particiones: {df.rdd.getNumPartitions()}")

# Reparticionar
df_repart = df.repartition(8, "category")

# Coalesce (reduce particiones sin shuffle)
df_coalesce = df.coalesce(2)

# Particionamiento al escribir
df.write \
    .partitionBy("category", "year") \
    .mode("overwrite") \
    .parquet("data/techstore_optimized/")
```

2.3.3 Caching y Persistencia

```
from pyspark.storagelevel import StorageLevel

# Cache en memoria (para datos reutilizados)
df_cached = df.filter(col("amount") > 0).cache()
df_cached.count() # materializa el cache

# Storage levels
niveles = {
    "MEMORY_ONLY": "Solo memoria (default)",
    "MEMORY_AND_DISK": "Memoria, luego disco",
    "DISK_ONLY": "Solo disco",
    "OFF_HEAP": "Fuera del heap de JVM"
}

# Persistir con nivel específico
df_large = df.persist(StorageLevel.MEMORY_AND_DISK)

# Liberar
df_cached.unpersist()
```

2.4 Spark UI: Diagnóstico y Tuning

```
# Configuración para ver Spark UI
spark = SparkSession.builder \
    .appName("TechStore-Optimized") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.sql.adaptive.coalescePartitions.enabled", "true") \
    .config("spark.sql.adaptive.skewJoin.enabled", "true") \
    .config("spark.sql.autoBroadcastJoinThreshold", "100MB") \
    .getOrCreate()
```

Métricas clave en Spark UI:

- **Shuffle Read/Write:** Datos movidos entre nodos
- **Stage Duration:** Tiempo por etapa
- **GC Time:** Tiempo de garbage collection
- **Spill:** Datos escritos a disco por falta de memoria

2.5 Ejercicios

1. **RDD:** Crea un RDD con 1000 números aleatorios y calcula media, mediana y desviación.
2. **DataFrame:** Lee `techstore_transactions`` y calcula el top 10 productos por ingresos totales.
3. **SQL:** Usa Spark SQL para encontrar los clientes con mayor gasto acumulado en 2025.
4. **Optimización:** Mide la diferencia de rendimiento entre `df.filter().groupBy().agg()` con y sin cache.
5. **Particionamiento:** Genera un dataset particionado por `category`` y `year`` y explica el beneficio.

Capítulo 3: Procesamiento de Datos a Escala

3.1 ETL con PySpark

El proceso ETL (Extract, Transform, Load) es el pan de cada día del ingeniero de datos. Con PySpark, escalamos ETL a volúmenes masivos.

3.1.1 Extract: Lectura desde Múltiples Fuentes

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import input_file_name, col

spark = SparkSession.builder.appName("TechStore-ETL").getOrCreate()

# Múltiples archivos con glob pattern
df_csv = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("data/raw/techstore_*.csv")

# Múltiples fuentes
fuentes = {
    "parquet": spark.read.parquet("data/transactions/"),
    "json": spark.read.json("data/events/"),
    "jdbc": spark.read.format("jdbc") \
        .option("url", "jdbc:postgresql://localhost/techstore") \
        .option("dbtable", "customers") \
        .option("user", "admin") \
        .option("password", "*****").load(),
    "bigquery": spark.read.format("bigquery") \
        .option("table", "techstore.users").load()
}
```

3.1.2 Transform: Limpieza y Enriquecimiento

```
# Pipeline de transformación completo
def transformar_ventas(df):
    return df \
        .filter(col("amount").isNotNull() & (col("amount") > 0)) \
        .filter(col("customer_id").isNotNull()) \
```

```

.withColumn("fecha", col("timestamp").cast("date")) \
.withColumn("año", F.year("timestamp")) \
.withColumn("mes", F.month("timestamp")) \
.withColumn("trimestre", F.quarter("timestamp")) \
.withColumn("iva", col("amount") * 0.21) \
.withColumn("total_con_iva", col("amount") + col("iva")) \
.withColumn("rango_precio",
             when(col("amount") < 50, "Económico")
             .when(col("amount") < 200, "Estándar")
             .otherwise("Premium")
            ) \
.dropDuplicates(["transaction_id", "customer_id"])

ventas_limpias = transformar_ventas(df)

```

3.1.3 Load: Escritura Optimizada

```

# Escritura con múltiples formatos
formatos = {
    "parquet": {
        "path": "data/warehouse/ventas/",
        "opciones": {"compression": "snappy"}
    },
    "delta": {
        "path": "data/lake/ventas/",
        "opciones": {"delta.autoOptimize": "true"}
    },
    "iceberg": {
        "path": "data/catalog/ventas/",
        "opciones": {"write.format": "parquet"}
    }
}

for fmt, conf in formatos.items():
    ventas_limpias.write \
        .mode("overwrite") \
        .partitionBy("año", "mes") \
        .option("compression", "snappy") \
        .format(fmt) \
        .save(conf["path"])

```

3.2 Manejo de Datos Semi-estructurados

3.2.1 Trabajando con JSON

```

from pyspark.sql.functions import from_json, get_json_object, json_tuple, schema_of_json
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, ArrayType

# Eventos TechStore en formato JSON
eventos_json = spark.read.text("data/events/raw/")

```

```

# Schema inferido o definido manualmente
schema = StructType([
    StructField("event_id", StringType()),
    StructField("user_id", StringType()),
    StructField("event_type", StringType()),
    StructField("product_id", StringType()),
    StructField("timestamp", StringType()),
    StructField("metadata", StructType([
        StructField("source", StringType()),
        StructField("session_id", StringType()),
        StructField("device", StringType())
    ]))
])

eventos_estructurados = eventos_json \
    .select(from_json(col("value"), schema).alias("event")) \
    .select("event.*")

# Anidamiento
eventos_con_dispositivo = eventos_estructurados \
    .select(
        "event_id", "user_id", "event_type",
        "metadata.device",
        "metadata.session_id"
    )

```

3.3 Broadcast Joins y Optimización

3.3.1 Tipos de Join en Spark

```

# Dimensiones pequeñas → Broadcast Join
productos = spark.read.parquet("data/dim_productos/") # pequeño
df_broadcast = ventas.join(
    broadcast(productos),
    "product_id",
    "left"
)
# Spark transmite productos a todos los workers

# Tablas grandes → Sort-Merge Join
df_largo = df_ventas.join(
    df_clientes, # también grande
    "customer_id",
    "inner"
)
# Spark: shuffle + sort + merge

# Bucket join (pre-particionado)
ventas_bucket = df_ventas.bucketBy(16, "customer_id") \
    .saveAsTable("ventas_bucket")
clientes_bucket = df_clientes.bucketBy(16, "customer_id") \
    .saveAsTable("clientes_bucket")

```

```
# No necesita shuffle
```

3.3.2 Skew Join Handling

```
# Detectar skew
skew = df.groupBy("customer_id").count() \
    .select(max("count"), avg("count"), stddev("count"))
skew.show()

# Si hay clientes con muchísimas transacciones

# Solución 1: Saltar skew (Spark 3.x automático)
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

# Solución 2: Salting manual
from pyspark.sql.functions import rand, concat, lit

df_salted = df_ventas.withColumn(
    "salt",
    (rand() * 10).cast("int")
)
clientes_salted = df_clientes.withColumn(
    "salt_array",
    F.array([lit(i) for i in range(10)])
).select(
    "*",
    F.explode("salt_array").alias("salt")
)

join_salted = df_salted.join(
    clientes_salted,
    ["customer_id", "salt"]
)
```

3.4 Catálogo TechStore Optimizado

```
# Crear una tabla optimizada con particionamiento y bucketing
spark.sql("""
    CREATE TABLE IF NOT EXISTS techstore.catalog (
        transaction_id STRING,
        customer_id INT,
        product_id INT,
        amount DECIMAL(10,2),
        quantity INT,
        timestamp TIMESTAMP,
        category STRING,
        year INT,
        month INT
    )
    USING parquet
    PARTITIONED BY (year, month)
    CLUSTERED BY (customer_id) INTO 32 BUCKETS
""")
```

```

        LOCATION 'data/techstore_catalog/'
    """)

# Insertar datos optimizados
ventas_limpias.write \
    .mode("overwrite") \
    .insertInto("techstore.catalog")

# Consultas ultra rápidas
spark.sql("""
    SELECT category, SUM(amount) as total
    FROM techstore.catalog
    WHERE year = 2025 AND customer_id = 42
    GROUP BY category
""").show()

```

3.5 Ejercicios

1. **ETL Pipeline:** Construye un ETL que lea 3 fuentes de TechStore, las limpie y las unifique en una tabla particionada.
2. **Broadcast Join:** Compara rendimiento de join normal vs broadcast join con una tabla de 10k productos.
3. **Skew:** Simula datos skewed donde el 10% de clientes generan el 90% de transacciones y optimiza el join.
4. **JSON:** Procesa eventos TechStore en JSON anidado y extrae métricas por sesión y dispositivo.
5. **Catálogo:** Diseña un catálogo particionado por fecha y bucketeado por customer_id para consultas rápidas.

Capítulo 4: Datos en Streaming

4.1 Procesamiento en Tiempo Real

El big data no es solo volumen; también es velocidad. Los datos en streaming llegan continuamente y deben procesarse en ventanas de tiempo.

4.1.1 Batch vs Streaming

```
# Batch: procesamiento periódico
df_batch = spark.read.parquet("data/dia_2025_01_15/")
resultado = df_batch.groupBy("category").sum("amount")

# Streaming: procesamiento continuo
df_stream = spark \
    .readStream \
    .format("rate") \
    .option("rowsPerSecond", "1000") \
    .load()
```

4.1.2 Structured Streaming

Spark Structured Streaming trata los streams como DataFrames infinitos:

```
from pyspark.sql.functions import window, col, sum

# Definir el stream de entrada
eventos_stream = spark \
    .readStream \
    .schema("event_id STRING, customer_id INT, amount DOUBLE, event_ts TIMESTAMP") \
    .option("maxFilesPerTrigger", 1) \
    .parquet("data/events/streaming/")

# Transformaciones en streaming (igual que batch)
metricas_stream = eventos_stream \
    .withWatermark("event_ts", "10 minutes") \
    .groupBy(
        window("event_ts", "5 minutes"),
        "customer_id"
    ) \
    .agg(sum("amount").alias("gasto_5min"))

# Salida
query = metricas_stream \
```

```

        .writeStream \
        .queryName("ventas_tiempo_real") \
        .outputMode("append") \
        .format("console") \
        .trigger(processingTime="10 seconds") \
        .start()

query.awaitTermination(timeout=60)

```

4.2 Kafka: El Sistema Nervioso del Streaming

4.2.1 Conceptos de Kafka

```

conceptos_kafka = {
    "Producer": "Publica mensajes en topics",
    "Consumer": "Lee mensajes de topics",
    "Topic": "Canal de mensajes categorizado",
    "Partition": "División de un topic para paralelismo",
    "Broker": "Servidor Kafka",
    "Offset": "Posición del mensaje en la partición",
    "Consumer Group": "Grupo de consumidores que balancean carga"
}

```

4.2.2 Consumir desde Kafka con Spark

```

# Configuración del consumidor Spark-Kafka
kafka_df = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "techstore_events") \
    .option("startingOffsets", "latest") \
    .option("failOnDataLoss", "false") \
    .load()

# Parsear mensajes JSON
from pyspark.sql.functions import from_json, col

schema_stream = "event_id STRING, event_type STRING, product_id INT, ts TIMESTAMP"

eventos = kafka_df \
    .select(from_json(col("value").cast("string"), schema_stream).alias("event")) \
    .select("event.*")

# Eventos en tiempo real
eventos_tiempo_real = eventos \
    .withWatermark("ts", "1 minute") \
    .groupBy(
        window("ts", "1 minute"),
        "event_type"
    ) \

```

```
.count() \  
.orderBy("window")
```

4.3 Ventanas de Tiempo y Watermarks

```
from pyspark.sql.functions import window, current_timestamp, expr  
  
# Tipos de ventanas  
ventanas = {  
    "Tumbling": "Ventanas fijas sin solapamiento [0-5, 5-10, 10-15]",  
    "Sliding": "Ventanas con solapamiento [0-5, 2-7, 4-9]",  
    "Session": "Ventanas basadas inactividad [gap=10min]"  
}  
  
# Tumbling window cada 5 minutos  
ventas_5min = eventos_stream \  
    .withWatermark("event_ts", "5 minutes") \  
    .groupBy(  
        window("event_ts", "5 minutes"),  
        "product_id"  
    ) \  
    .agg(  
        sum("amount").alias("ingresos"),  
        count("*").alias("ventas")  
    )  
  
# Sliding window cada 2 minutos, con ventana de 10  
ventas_sliding = eventos_stream \  
    .groupBy(  
        window("event_ts", "10 minutes", "2 minutes"),  
        "category"  
    ) \  
    .agg(sum("amount").alias("total"))  
  
# Session window con gap de 5 minutos  
comportamiento = eventos_stream \  
    .groupBy(  
        session_window("event_ts", "5 minutes"),  
        "customer_id"  
    ) \  
    .agg(count("*").alias("eventos_sesion"))  
  
# Watermark: tolerancia a datos tardíos  
# Si watermark = 10 minutos, datos con ts < max(ts) - 10min se descartan
```

4.4 Output Modes y Sinks

```
# Output modes  
modos = {  
    "append": "Solo nuevas filas (agregaciones sin estado)",
```

```

    "complete": "Todo el resultado (agregaciones con estado)",
    "update": "Solo filas actualizadas (similar a complete pero más eficiente)"
}

# Sinks de salida
sinks = {
    "console": query.writeStream.format("console"),
    "memory": query.writeStream.format("memory").queryName("tabla_stream"),
    "parquet": query.writeStream.format("parquet")
        .option("path", "data/stream_output/")
        .option("checkpointLocation", "data/checkpoints/parquet/"),
    "kafka": query.writeStream.format("kafka")
        .option("kafka.bootstrap.servers", "localhost:9092")
        .option("topic", "techstore_metrics"),
    "foreach": query.writeStream.foreach(procesador_personalizado)
}

# Checkpointing para recuperación
query = metricas_stream \
    .writeStream \
    .option("checkpointLocation", "data/checkpoints/stream_ventas/") \
    .trigger(processingTime="30 seconds") \
    .foreachBatch(procesar_lote) \
    .start()

def procesar_lote(df, epoch_id):
    print(f"Procesando lote {epoch_id} con {df.count()} registros")
    df.write.mode("append").parquet(f"data/stream/{epoch_id}/")

```

4.5 Ejercicios

1. **Streaming Básico:** Lee un stream de eventos TechStore y cuenta eventos por tipo cada minuto.
2. **Kafka:** Configura un productor Kafka que publique eventos de navegación y un consumidor Spark que los procese.
3. **Watermark:** Implementa una ventana de 5 minutos con watermark de 2 minutos y explica el comportamiento con datos tardíos.
4. **Output Mode:** Compara los 3 output modes para una agregación ``groupBy("product_id").count()`.`
5. **Checkpointing:** Diseña un pipeline streaming con checkpointing que pueda recuperarse de una caída sin perder datos.

Checkpoint 1: Pipeline de Big Data con TechStore

Objetivo

Construir un pipeline completo de big data que procese el dataset TechStore a escala, combinando procesamiento batch y streaming.

Requisitos

- PySpark configurado (local o cluster)
- Dataset TechStore generado (mínimo 1M transacciones)
- Kafka (opcional, para versión streaming)
- 30-45 minutos de ejecución

Dataset

Archivo	Registros	Formato
`techstore_transactions.parquet`	1,000,000+	Parquet
`techstore_customers.parquet`	100,000	Parquet
`techstore_products.parquet`	5,000	Parquet
`techstore_events_streaming/`	∞ (streaming)	JSON por línea

Pipeline

Fase 1: Data Lake Organizado

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, avg, count, when, year, month
import pyspark.sql.functions as F

spark = SparkSession.builder \
    .appName("Checkpoint1-BigData") \
    .config("spark.sql.adaptive.enabled", "true") \
    .getOrCreate()

# 1. Leer datos crudos
transacciones = spark.read.parquet("data/techstore_transactions/")
```

```

clientes = spark.read.parquet("data/techstore_customers/")
productos = spark.read.parquet("data/techstore_products/")

# 2. Organizar en zonas del data lake
# Bronze: datos crudos (ya están en raw/)
# Silver: datos limpios y enriquecidos
silver_ventas = transacciones \
    .filter(col("amount").isNotNull() & (col("amount") > 0)) \
    .join(broadcast(productos.select("product_id", "category", "product_name")), "product_id", "left") \
    .join(broadcast(clientes.select("customer_id", "segment", "region")), "customer_id", "left") \
    .withColumn("year", year("timestamp")) \
    .withColumn("month", month("timestamp"))

silver_ventas.write \
    .mode("overwrite") \
    .partitionBy("year", "month") \
    .parquet("data/silver/ventas/")

```

Fase 2: Capa Gold (Métricas de Negocio)

```

# Gold: agregaciones listas para consumo
spark.sql("""
    CREATE OR REPLACE TEMP VIEW gold_ventas_diarias AS
    SELECT
        date_trunc('day', timestamp) as dia,
        category,
        region,
        segment,
        COUNT(DISTINCT customer_id) as clientes_activos,
        COUNT(*) as transacciones,
        ROUND(SUM(amount), 2) as ingresos,
        ROUND(AVG(amount), 2) as ticket_promedio
    FROM silver_ventas
    GROUP BY dia, category, region, segment
""")

# Métricas de rendimiento de producto
spark.sql("""
    CREATE OR REPLACE TEMP VIEW gold_top_productos AS
    SELECT
        product_id,
        product_name,
        category,
        COUNT(*) as veces_vendido,
        ROUND(SUM(amount), 2) as ingresos_totales,
        ROUND(AVG(amount), 2) as precio_promedio,
        RANK() OVER (PARTITION BY category ORDER BY SUM(amount) DESC) as rank_categoria
    FROM silver_ventas
    GROUP BY product_id, product_name, category
    HAVING COUNT(*) > 10
""")

```

Fase 3: Pipeline Streaming

```
# Leer streaming de eventos de navegación
eventos = spark \
    .readStream \
    .schema("event_id STRING, customer_id INT, product_id INT, event_type STRING, ts TIMESTAMP") \
    .option("maxFilesPerTrigger", 1) \
    .json("data/events_streaming/")

# Combinar con datos batch para enriquecer
productos_broadcast = broadcast(productos)

eventos_enriquecidos = eventos \
    .join(productos_broadcast, "product_id", "left") \
    .withWatermark("ts", "5 minutes")

# Métricas en tiempo real
metricas_stream = eventos_enriquecidos \
    .groupBy(
        F.window("ts", "5 minutes"),
        "category",
        "event_type"
    ) \
    .agg(
        count("*").alias("total_eventos"),
        countDistinct("customer_id").alias("usuarios_unicos")
    )

# ForeachBatch para escribir en gold
def escribir_gold(df, epoch_id):
    df.write \
        .mode("append") \
        .partitionBy("category") \
        .parquet(f"data/gold/streaming/")

query = metricas_stream \
    .writeStream \
    .foreachBatch(escribir_gold) \
    .option("checkpointLocation", "data/checkpoints/cp1/") \
    .trigger(processingTime="30 seconds") \
    .start()

query.awaitTermination(timeout=120)
```

Validación

```
# Verificar pipeline batch
print("=== Validación Silver ===")
print(f"Registros silver: {spark.read.parquet('data/silver/ventas/').count():,}")

print("\n=== Validación Gold ===")
print(spark.sql("SELECT * FROM gold_ventas_diarias ORDER BY ingresos DESC LIMIT 10").toPandas())
```

```
print("\n=== Top Productos por Categoría ===")
print(spark.sql("""
    SELECT category, product_name, ingresos_totales, rank_categoria
    FROM gold_top_productos
    WHERE rank_categoria <= 3
    ORDER BY category, rank_categoria
    """).toPandas())
```

Entregables

1. Código del pipeline completo (este script)
2. Data Lake en 3 capas: Bronze Silver Gold
3. Pipeline streaming funcional con checkpointing
4. Tablas Gold listas para BI y ML

Bonus: Dashboard en Tiempo Real

```
import streamlit as st
import pandas as pd
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Dashboard-Streaming").getOrCreate()

st.title("TechStore - Dashboard Tiempo Real")

# Auto-refresh cada 5 segundos
while True:
    df_gold = spark.read.parquet("data/gold/streaming/")
    pdf = df_gold.groupBy("category").sum("total_eventos").toPandas()
    st.bar_chart(pdf.set_index("category"))
    time.sleep(5)
```

Criterios de Evaluación

Criterio	Puntos
Pipeline batch funcional	25
Capas Bronze/Silver/Gold correctas	25
Pipeline streaming con watermark	25
Checkpointing y recuperación	15
Dashboard en tiempo real (bonus)	10

¡Felicidades! Has construido tu primer pipeline de big data completo con TechStore.

Capítulo 5: Introducción al Machine Learning

5.1 El Pipeline de ML

El machine learning no es magia; es un proceso estructurado. El pipeline clásico consta de:

```
graph LR
  A[Datos] --> B[Preprocesamiento]
  B --> C[Train/Test Split]
  C --> D[Entrenamiento]
  D --> E[Evaluación]
  E --> F[Despliegue]
  F --> G[Monitoreo]
  G --> A
```

5.1.1 Tipos de Aprendizaje

```
tipos = {
  "Supervisado": "Etiquetas conocidas (regresión, clasificación)",
  "No supervisado": "Sin etiquetas (clustering, reducción de dimensionalidad)",
  "Semi-supervisado": "Pocas etiquetas, muchos datos sin etiquetar",
  "Refuerzo": "Agente aprende por recompensas (RL)"
}
```

5.2 Scikit-learn: La Biblioteca Esencial

5.2.1 API Consistente

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import pandas as pd
import numpy as np

# Cargar datos TechStore
df = pd.read_parquet("data/silver/ventas/")
print(f"Dataset: {df.shape[0]:,} registros, {df.shape[1]} features")
```

```

# Separar features y target
X = df.drop(["ingresos", "transaction_id", "timestamp"], axis=1)
y = df["ingresos"]

# Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Train: {len(X_train):,} | Test: {len(X_test):,}")

```

5.2.2 Preprocesamiento

```

# Definir tipos de columnas
num_features = ["precio_unitario", "cantidad", "descuento", "edad_cliente"]
cat_features = ["categoria", "region", "segmento", "metodo_pago"]

# Transformadores
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_features),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_features)
    ]
)

# Pipeline completo
modelo = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(n_estimators=100, random_state=42))
])

# Entrenar
modelo.fit(X_train, y_train)

```

5.3 Feature Engineering

5.3.1 Creación de Features

```

# Features temporales
df["hora"] = pd.to_datetime(df["timestamp"]).dt.hour
df["dia_semana"] = pd.to_datetime(df["timestamp"]).dt.dayofweek
df["es_fin_semana"] = df["dia_semana"].isin([5, 6]).astype(int)
df["mes"] = pd.to_datetime(df["timestamp"]).dt.month

# Features agregadas por cliente
features_cliente = df.groupby("customer_id").agg(
    freq_compra=("transaction_id", "count"),
    ticket_avg=("amount", "mean"),
    ticket_max=("amount", "max"),
    antiguedad_dias=("timestamp", lambda x: (x.max() - x.min()).days)
).reset_index()

```

```
# Features de producto
features_producto = df.groupby("product_id").agg(
    ventas_totales=("amount", "sum"),
    veces_vendido=("transaction_id", "count"),
    precio_promedio=("amount", "mean")
).reset_index()
```

5.3.2 Selección de Features

```
from sklearn.feature_selection import SelectKBest, f_regression, mutual_info_regression

# Selección basada en correlación
selector = SelectKBest(score_func=f_regression, k=10)
X_selected = selector.fit_transform(X_train, y_train)

# Importancia del modelo
modelo.fit(X_train, y_train)
importancias = pd.DataFrame({
    "feature": X_train.columns,
    "importance": modelo.named_steps["regressor"].feature_importances_
}).sort_values("importance", ascending=False)

print(importancias.head(10))
```

5.4 Train/Test/Validation Split

```
from sklearn.model_selection import train_test_split, cross_val_score

# Split triple para modelos complejos
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)

print(f"Train: {len(X_train):,} | Val: {len(X_val):,} | Test: {len(X_test):,}")

# Cross-validation
scores = cross_val_score(modelo, X_train, y_train, cv=5, scoring="r2")
print(f"CV R2: {scores.mean():.3f} +/- {scores.std():.3f}")
```

5.5 Métricas Fundamentales

```
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score,
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)
```

```
# Para regresión: predicción de ingresos
metricas_regresion = {
    "MSE": mean_squared_error(y_test, y_pred),
    "RMSE": np.sqrt(mean_squared_error(y_test, y_pred)),
    "MAE": mean_absolute_error(y_test, y_pred),
    "R2": r2_score(y_test, y_pred),
    "MAPE": np.mean(np.abs((y_test - y_pred) / y_test)) * 100
}
```

5.6 Ejercicios

1. **Pipeline:** Construye un pipeline completo para predecir ingresos de TechStore con preprocessing y 3 modelos diferentes.
2. **Feature Engineering:** Crea 5 features nuevas a partir de los datos de TechStore y mide su impacto en R2.
3. **Cross-Validation:** Compara CV=5 vs CV=10 en términos de tiempo y estabilidad de métricas.
4. **Métricas:** Para un modelo de clasificación (cliente premium vs básico), calcula precisión, recall y F1.
5. **Overfitting:** Demuestra overfitting entrenando un árbol de decisión sin limitar profundidad vs con max_depth=5.

Capítulo 6: Aprendizaje Supervisado

6.1 Regresión

6.1.1 Regresión Lineal

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Modelo base
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)

print(f"R2: {r2_score(y_test, y_pred):.3f}")
print(f"RMSE: ${np.sqrt(mean_squared_error(y_test, y_pred)):.2f}")

# Coeficientes (interpretabilidad)
coeficientes = pd.DataFrame({
    "feature": num_features,
    "coef": lr.coef_[1:len(num_features)]
}).sort_values("coef", key=abs, ascending=False)

print("Features más importantes:")
print(coeficientes.head(5))

# Ridge (L2) vs Lasso (L1)
ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.01)

ridge.fit(X_train, y_train)
lasso.fit(X_train, y_train)

print(f"Ridge R2: {r2_score(y_test, ridge.predict(X_test)):.3f}")
print(f"Lasso R2: {r2_score(y_test, lasso.predict(X_test)):.3f}")
print(f"Features usadas por Lasso: {np.sum(lasso.coef_ != 0)}/{len(lasso.coef_)}")
```

6.1.2 Árboles de Decisión

```
from sklearn.tree import DecisionTreeRegressor, plot_tree
import matplotlib.pyplot as plt
```

```

# Árbol simple (interpretable)
dt = DecisionTreeRegressor(max_depth=3, min_samples_leaf=50, random_state=42)
dt.fit(X_train, y_train)

print(f"R2 (train): {dt.score(X_train, y_train):.3f}")
print(f"R2 (test): {dt.score(X_test, y_test):.3f}")

# Visualizar el árbol
plt.figure(figsize=(20, 10))
plot_tree(dt, feature_names=X_train.columns, filled=True, fontsize=10)
plt.savefig("img/arbol_decision.png", dpi=150)

# Feature importance
importancias = pd.DataFrame({
    "feature": X_train.columns,
    "importance": dt.feature_importances_
}).sort_values("importance", ascending=False)

```

6.1.3 Random Forest

```

from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from xgboost import XGBRegressor

# Random Forest
rf = RandomForestRegressor(
    n_estimators=200,
    max_depth=15,
    min_samples_leaf=10,
    n_jobs=-1,
    random_state=42
)
rf.fit(X_train, y_train)

# Gradient Boosting
gbr = GradientBoostingRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=5,
    random_state=42
)
gbr.fit(X_train, y_train)

# XGBoost (el rey de los datos tabulares)
xgb = XGBRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=7,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

```

```
xgb.fit(X_train, y_train)

# Comparación
modelos = {
    "Random Forest": rf,
    "Gradient Boosting": gbr,
    "XGBoost": xgb
}

for nombre, modelo in modelos.items():
    y_pred = modelo.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)
    print(f"{nombre:20s} | RMSE: ${rmse:.2f} | R2: {r2:.3f}")
```

6.2 Clasificación

6.2.1 Regresión Logística

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

# Crear target binario: cliente premium (>$1000 gasto total)
df["es_premium"] = (df.groupby("customer_id")["amount"].transform("sum") > 1000).astype(int)

y_clf = df["es_premium"]
X_clf = df.drop(["es_premium", "transaction_id", "timestamp", "amount"], axis=1)

X_c_train, X_c_test, y_c_train, y_c_test = train_test_split(
    X_clf, y_clf, test_size=0.2, random_state=42
)

# Regresión logística
log_reg = LogisticRegression(class_weight="balanced", max_iter=1000)
log_reg.fit(X_c_train, y_c_train)
y_c_pred = log_reg.predict(X_c_test)
y_c_proba = log_reg.predict_proba(X_c_test)[:, 1]

print(classification_report(y_c_test, y_c_pred))
print(f"AUC-ROC: {roc_auc_score(y_c_test, y_c_proba):.3f}")
```

6.2.2 Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

rf_clf = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    class_weight="balanced",
    random_state=42
)
```

```

rf_clf.fit(X_c_train, y_c_train)
y_c_pred_rf = rf_clf.predict(X_c_test)

print("Random Forest Classifier:")
print(classification_report(y_c_test, y_c_pred_rf))

# Matriz de confusión
cm = confusion_matrix(y_c_test, y_c_pred_rf)
print(f"\nMatriz de Confusión:")
print(f"VN: {cm[0,0]:,} | FP: {cm[0,1]:,}")
print(f"FN: {cm[1,0]:,} | VP: {cm[1,1]:,}")

```

6.2.3 XGBoost Classifier

```

from xgboost import XGBClassifier

xgb_clf = XGBClassifier(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=6,
    scale_pos_weight=(y_c_train == 0).sum() / (y_c_train == 1).sum(),
    random_state=42
)
xgb_clf.fit(X_c_train, y_c_train)

# Feature importance
importance_df = pd.DataFrame({
    "feature": X_c_train.columns,
    "importance": xgb_clf.feature_importances_
}).sort_values("importance", ascending=False)

print("Top 10 features predictivas:")
print(importance_df.head(10))

```

6.3 Ejercicios

1. **Regresión:** Predice el ticket promedio por cliente usando Random Forest y optimiza los hiperparámetros.
2. **Clasificación:** Crea un modelo que prediga si un producto será "top seller" (>100 unidades) usando XGBoost.
3. **Comparación:** Compara LinearRegression, Ridge, Lasso, Random Forest y XGBoost en el mismo dataset. ¿Cuál gana?
4. **Interpretación:** Usa SHAP para explicar las predicciones del mejor modelo de clasificación.
5. **Desequilibrio:** Crea un dataset desbalanceado (95% no-premium, 5% premium) y prueba técnicas de balanceo (SMOTE, class_weight).

Capítulo 7: Aprendizaje No Supervisado

7.1 Clustering con K-Means

7.1.1 Segmentación de Clientes

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

# Preparar datos de clientes TechStore
clientes = pd.read_parquet("data/silver/clientes/")

features_segmentacion = [
    "frecuencia_compra", "ticket_promedio", "gasto_total",
    "antiguedad_dias", "categorias_compradas", "dias_ultima_compra"
]

X_seg = clientes[features_segmentacion].dropna()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_seg)

# Encontrar K óptimo (método del codo)
inercias = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inercias.append(km.inertia_)

# Gráfico del codo
plt.figure(figsize=(8, 5))
plt.plot(range(2, 11), inercias, "bo-")
plt.xlabel("K")
plt.ylabel("Inercia")
plt.title("Método del Codo para K-Means")
plt.savefig("img/codo_kmeans.png")
```

7.1.2 Interpretación de Clusters

```

# Aplicar K-Means con K=4 (óptimo)
k = 4
km = KMeans(n_clusters=k, random_state=42, n_init=10)
clientes["cluster"] = km.fit_predict(X_scaled)

# Perfil de cada cluster
perfil = clientes.groupby("cluster")[features_segmentacion].mean()
perfil["count"] = clientes.groupby("cluster").size()
perfil["porcentaje"] = (perfil["count"] / len(clientes) * 100).round(1)

print("=== Perfil de Clusters ===")
for cluster in range(k):
    print(f"\nCluster {cluster} ({perfil.loc[cluster, 'porcentaje']:.1f}%)")
    for feat in features_segmentacion:
        val = perfil.loc[cluster, feat]
        print(f"  {feat}: {val:.2f}")

# Visualización 2D con PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(10, 8))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clientes["cluster"],
                      cmap="viridis", alpha=0.6)
plt.colorbar(scatter)
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]:.1%})")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]:.1%})")
plt.title("Segmentación de Clientes TechStore")
plt.savefig("img/segmentacion_clientes.png")

# Nombres de segmentos
segmentos = {
    0: "Premium Fieles - Alto gasto, alta frecuencia",
    1: "Compradores Ocasionales - Gasto medio, baja frecuencia",
    2: "Nuevos - Poca antigüedad, compras recientes",
    3: "En Riesgo - Alta antigüedad, sin compras recientes"
}

```

7.2 Sistemas de Recomendación

7.2.1 Filtrado Colaborativo

```

from sklearn.metrics.pairwise import cosine_similarity
from scipy.sparse import csr_matrix

# Matriz usuario-producto
matriz_compras = df.groupby(["customer_id", "product_id"])["amount"].sum().unstack(fill_value=0)
matriz_sparse = csr_matrix(matriz_compras.values)

# Similitud entre usuarios
sim_usuarios = cosine_similarity(matriz_sparse)

```

```

# Sistema de recomendación simple
def recomendar_para_usuario(user_id, n_recomendaciones=5):
    idx = list(matriz_compras.index).index(user_id)
    similar_users = sim_usuarios[idx].argsort()[-6:-1][::-1]

    productos_comprados = set(matriz_compras.columns[matriz_compras.iloc[idx] > 0])
    recomendaciones = {}

    for sim_idx in similar_users:
        productos_vecino = matriz_compras.columns[matriz_compras.iloc[sim_idx] > 0]
        nuevos = set(productos_vecino) - productos_comprados
        for prod in nuevos:
            recomendaciones[prod] = recomendaciones.get(prod, 0) + sim_usuarios[idx][sim_idx]

    return sorted(recomendaciones.items(), key=lambda x: x[1], reverse=True)[:n_recomendaciones]

# Ejemplo
user_ejemplo = 42
recomendaciones = recomendar_para_usuario(user_ejemplo)
print(f"Recomendaciones para usuario {user_ejemplo}:")
for prod, score in recomendaciones:
    print(f" → Producto {prod}: score {score:.3f}")

```

7.2.2 Filtrado Basado en Contenido

```

from sklearn.feature_extraction.text import TfidfVectorizer

# Productos con descripciones
productos["descripcion_completa"] = (
    productos["product_name"] + " " +
    productos["category"] + " " +
    productos["description"] + " " +
    productos["keywords"]
)

# TF-IDF sobre descripciones
tfidf = TfidfVectorizer(max_features=1000, stop_words="spanish")
matriz_tfidf = tfidf.fit_transform(productos["descripcion_completa"])

# Similitud entre productos
sim_productos = cosine_similarity(matriz_tfidf)

def recomendar_similares(producto_id, n=5):
    idx = productos[productos["product_id"] == producto_id].index[0]
    scores = list(enumerate(sim_productos[idx]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)[1:n+1]

    print(f"Producto: {productos.iloc[idx]['product_name']}")
    print("Productos similares:")
    for i, score in scores:
        p = productos.iloc[i]
        print(f" → {p['product_name']} ({p['category']}) - {score:.2%} similitud")

```

```
recomendar_similares(1234)
```

7.3 Reducción de Dimensionalidad

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# PCA para reducir dimensionalidad
pca = PCA(n_components=0.95) # mantener 95% de varianza
X_pca = pca.fit_transform(X_scaled)

print(f"Dimensiones originales: {X_scaled.shape[1]}")
print(f"Dimensiones PCA (95% varianza): {X_pca.shape[1]}")
print(f"Varianza explicada por componente:")
for i, var in enumerate(pca.explained_variance_ratio_):
    print(f"  PC{i+1}: {var:.1%}")

# t-SNE para visualización
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
X_tsne = tsne.fit_transform(X_scaled)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clientes["cluster"], cmap="viridis", alpha=0.6)
plt.title("PCA")

plt.subplot(1, 2, 2)
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=clientes["cluster"], cmap="viridis", alpha=0.6)
plt.title("t-SNE")

plt.savefig("img/pca_vs_tsne.png")
```

7.4 Ejercicios

1. **Segmentación:** Aplica K-Means con K=3, 4, 5 a los clientes TechStore. ¿Cuál segmentación es más interpretable?
2. **Recomendación:** Implementa un sistema híbrido que combine filtrado colaborativo y basado en contenido.
3. **PCA:** Reduce las features de transacciones TechStore a 2 componentes y visualiza. ¿Se pierde mucha información?
4. **t-SNE:** Compara t-SNE con PCA para visualizar los clusters de clientes. ¿Cuál es mejor?
5. **DBSCAN:** Aplica DBSCAN a los datos de clientes y compara los clusters con K-Means. ¿Qué diferencias encuentras?

Capítulo 8: Evaluación y Optimización de Modelos

8.1 Métricas de Evaluación

8.1.1 Para Regresión

```
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score,
    mean_absolute_percentage_error, explained_variance_score
)

def evaluar_regresion(y_true, y_pred, nombre="Modelo"):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    mape = mean_absolute_percentage_error(y_true, y_pred) * 100
    r2 = r2_score(y_true, y_pred)
    ev = explained_variance_score(y_true, y_pred)

    print(f"=== {nombre} ===")
    print(f"RMSE:  ${rmse:.2f}")
    print(f"MAE:  ${mae:.2f}")
    print(f"MAPE:  {mape:.1f}%")
    print(f"R2:   {r2:.3f}")
    print(f"EV:   {ev:.3f}")

    return {"RMSE": rmse, "MAE": mae, "MAPE": mape, "R2": r2}
```

8.1.2 Para Clasificación

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_curve, auc, roc_auc_score, confusion_matrix,
    classification_report, log_loss
)

def evaluar_clasificacion(y_true, y_pred, y_proba=None, nombre="Modelo"):
    print(f"=== {nombre} ===")

    accuracy = accuracy_score(y_true, y_pred)
```

```

precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

print(f"Accuracy: {accuracy:.3f}")
print(f"Precision: {precision:.3f}")
print(f"Recall: {recall:.3f}")
print(f"F1-Score: {f1:.3f}")

if y_proba is not None:
    auc = roc_auc_score(y_true, y_proba)
    print(f"AUC-ROC: {auc:.3f}")

# Matriz de confusión
cm = confusion_matrix(y_true, y_pred)
print(f"\nMatriz de Confusión:")
print(f"  VP: {cm[1,1]:,}  FP: {cm[0,1]:,}")
print(f"  FN: {cm[1,0]:,}  VN: {cm[0,0]:,}")

return {"accuracy": accuracy, "precision": precision,
        "recall": recall, "f1": f1}

```

8.2 Validación Cruzada

```

from sklearn.model_selection import (
    cross_val_score, cross_validate, KFold, StratifiedKFold,
    learning_curve, validation_curve
)
import matplotlib.pyplot as plt

# K-Fold para regresión
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(rf, X_train, y_train, cv=kf, scoring="r2")
print(f"R2 por fold: {scores}")
print(f"R2 promedio: {scores.mean():.3f} ± {scores.std():.3f}")

# Cross-validate completa
cv_results = cross_validate(
    rf, X_train, y_train,
    cv=5,
    scoring=["r2", "neg_mean_squared_error", "neg_mean_absolute_error"],
    return_train_score=True
)

print(f"Train R2: {cv_results['train_r2'].mean():.3f}")
print(f"Test R2: {cv_results['test_r2'].mean():.3f}")

# Curvas de aprendizaje
train_sizes, train_scores, test_scores = learning_curve(
    rf, X_train, y_train,
    cv=5, n_jobs=-1,

```

```

        train_sizes=np.linspace(0.1, 1.0, 10)
    )

    plt.figure(figsize=(10, 6))
    plt.plot(train_sizes, train_scores.mean(axis=1), "o-", label="Train")
    plt.plot(train_sizes, test_scores.mean(axis=1), "o-", label="Validation")
    plt.xlabel("Training examples")
    plt.ylabel("R2 Score")
    plt.legend()
    plt.title("Learning Curve - Random Forest")
    plt.savefig("img/learning_curve.png")

```

8.3 Hyperparameter Tuning

```

from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from scipy.stats import randint, uniform
import time

# Grid Search (exhaustivo)
param_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [10, 15, 20, None],
    "min_samples_leaf": [1, 5, 10]
}

grid_search = GridSearchCV(
    RandomForestRegressor(random_state=42),
    param_grid,
    cv=3,
    scoring="r2",
    n_jobs=-1,
    verbose=1
)

t0 = time.time()
grid_search.fit(X_train, y_train)
print(f"Grid Search: {time.time() - t0:.1f}s")

print(f"Mejores parámetros: {grid_search.best_params_}")
print(f"Mejor R2 (CV): {grid_search.best_score_:.3f}")

# Random Search (más eficiente)
param_dist = {
    "n_estimators": randint(100, 500),
    "max_depth": randint(5, 30),
    "min_samples_leaf": randint(1, 20),
    "max_features": ["sqrt", "log2", None],
    "bootstrap": [True, False]
}

random_search = RandomizedSearchCV(

```

```

RandomForestRegressor(random_state=42),
param_dist,
n_iter=50,
cv=3,
scoring="r2",
n_jobs=-1,
random_state=42
)

t0 = time.time()
random_search.fit(X_train, y_train)
print(f"\nRandom Search: {time.time() - t0:.1f}s")

print(f"Mejores parámetros: {random_search.best_params_}")
print(f"Mejor R2 (CV): {random_search.best_score_:.3f}")

```

8.4 Overfitting y Underfitting

```

# Detectar overfitting
def detectar_overfitting(modelo, X_train, y_train, X_test, y_test):
    train_score = modelo.score(X_train, y_train)
    test_score = modelo.score(X_test, y_test)
    diff = train_score - test_score

    print(f"Train R2: {train_score:.3f}")
    print(f"Test R2: {test_score:.3f}")
    print(f"Diferencia: {diff:.3f}")

    if diff > 0.1:
        print(" Posible OVERFITTING (diferencia > 0.1)")
    elif diff < -0.05:
        print(" Posible UNDERFITTING (test > train)")
    else:
        print(" Modelo balanceado")

# Regularización para combatir overfitting
print("\n=== Sin regularización ===")
dt_deep = DecisionTreeRegressor(max_depth=None, random_state=42)
dt_deep.fit(X_train, y_train)
detectar_overfitting(dt_deep, X_train, y_train, X_test, y_test)

print("\n=== Con regularización ===")
dt_reg = DecisionTreeRegressor(max_depth=8, min_samples_leaf=20, random_state=42)
dt_reg.fit(X_train, y_train)
detectar_overfitting(dt_reg, X_train, y_train, X_test, y_test)

# Curva de validación para max_depth
param_range = range(1, 21)
train_scores, test_scores = validation_curve(
    DecisionTreeRegressor(random_state=42),
    X_train, y_train,

```

```

    param_name="max_depth",
    param_range=param_range,
    cv=5, scoring="r2"
)

plt.figure(figsize=(10, 6))
plt.plot(param_range, train_scores.mean(axis=1), "o-", label="Train")
plt.plot(param_range, test_scores.mean(axis=1), "o-", label="Validation")
plt.axvline(x=8, color="r", linestyle="--", label="Optimal depth")
plt.xlabel("max_depth")
plt.ylabel("R2 Score")
plt.legend()
plt.title("Validation Curve - max_depth")
plt.savefig("img/validation_curve.png")

```

8.5 Ejercicios

1. **Grid Search:** Optimiza un XGBoost con GridSearch sobre learning_rate, n_estimators y max_depth.
2. **Curvas de Aprendizaje:** Genera curvas de aprendizaje para Random Forest con diferentes tamaños de dataset. ¿Cuándo deja de mejorar?
3. **Overfitting:** Entrena un modelo sin regularización y otro con regularización fuerte. Compara train/test scores.
4. **Validación Cruzada:** Implementa StratifiedKFold para clasificación y explica por qué es mejor que KFold simple.
5. **Métricas:** Para un problema de detección de fraude (1% de casos), ¿qué métrica es más importante? Justifica.

Checkpoint 2: Modelo Predictivo para TechStore

Objetivo

Construir, evaluar y optimizar un modelo de machine learning que prediga el valor de vida del cliente (CLV) en TechStore.

Dataset

Usa los datos procesados del Checkpoint 1 (capa Silver) y enriquece con features de clientes.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from xgboost import XGBRegressor
import matplotlib.pyplot as plt
import seaborn as sns

# Cargar datos
ventas = pd.read_parquet("data/silver/ventas/")
clientes = pd.read_parquet("data/silver/clientes/")
```

Feature Engineering

```
# Agregar features por cliente
clv_features = ventas.groupby("customer_id").agg(
    freq_total=("transaction_id", "count"),
    monto_total=("amount", "sum"),
    ticket_promedio=("amount", "mean"),
    ticket_max=("amount", "max"),
    ticket_std=("amount", "std"),
    antiguedad_dias=("timestamp", lambda x: (x.max() - x.min()).days),
    dias_ultima_compra=("timestamp", lambda x: (pd.Timestamp("2025-12-31") - x.max()).days),
    categorias_distintas=("category", "nunique"),
    productos_distintos=("product_id", "nunique"),
```

```

    devoluciones=("es_devolucion", "sum"),
    metodo_pago_frecuente=("metodo_pago", lambda x: x.mode().iloc[0] if not x.mode().empty else "unknown")
).reset_index()

# Merge con datos demográficos
clv = clv_features.merge(clientes, on="customer_id", how="left")

# Target: CLV = monto_total (valor de vida del cliente)
y = clv["monto_total"]
X = clv.drop(["customer_id", "monto_total"], axis=1)

# Separar numéricas y categóricas
num_features = X.select_dtypes(include=[np.number]).columns.tolist()
cat_features = X.select_dtypes(include=["object", "category"]).columns.tolist()

print(f"Features numéricas: {len(num_features)}")
print(f"Features categóricas: {len(cat_features)}")
print(f"Dataset: {len(X):,} clientes")

```

Pipeline de Modelado

```

# Preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_features),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_features)
    ]
)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Modelos a probar
modelos = {
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42),
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=100, random_state=42),
    "XGBoost": XGBRegressor(n_estimators=200, learning_rate=0.05, random_state=42)
}

resultados = []

for nombre, modelo in modelos.items():
    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("regressor", modelo)
    ])

    # Cross-validation
    scores = cross_val_score(pipeline, X_train, y_train, cv=5, scoring="r2")

```

```

# Entrenar
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

resultados.append({
    "modelo": nombre,
    "R2_CV": scores.mean(),
    "R2_CV_std": scores.std(),
    "R2_Test": r2_score(y_test, y_pred),
    "RMSE": np.sqrt(mean_squared_error(y_test, y_pred)),
    "MAE": mean_absolute_error(y_test, y_pred)
})

resultados_df = pd.DataFrame(resultados).sort_values("R2_Test", ascending=False)
print(resultados_df.to_string(index=False))

```

Optimización del Mejor Modelo

```

# Optimizar XGBoost
param_grid = {
    "regressor__n_estimators": [100, 200, 300],
    "regressor__learning_rate": [0.01, 0.05, 0.1],
    "regressor__max_depth": [4, 6, 8],
    "regressor__subsample": [0.7, 0.8, 1.0],
    "regressor__colsample_bytree": [0.7, 0.8, 1.0]
}

pipeline_xgb = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", XGBRegressor(random_state=42))
])

grid = GridSearchCV(
    pipeline_xgb,
    param_grid,
    cv=3,
    scoring="r2",
    n_jobs=-1,
    verbose=1
)

grid.fit(X_train, y_train)

print(f"Mejores parámetros: {grid.best_params_}")
print(f"Mejor R2 (CV): {grid.best_score_:.3f}")
print(f"R2 (Test): {r2_score(y_test, grid.predict(X_test)):.3f}")

# Feature importance
best_model = grid.best_estimator_
importancias = pd.DataFrame({

```

```

        "feature": num_features,
        "importance": best_model.named_steps["regressor"].feature_importances_[0:len(num_features)]
    }).sort_values("importance", ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(data=importancias.head(15), x="importance", y="feature")
plt.title("Top 15 Features para Predicción de CLV")
plt.tight_layout()
plt.savefig("img/feature_importance_clv.png")

```

Análisis de Errores

```

# Análisis de residuos
y_pred_best = best_model.predict(X_test)
residuos = y_test - y_pred_best

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.scatter(y_pred_best, y_test, alpha=0.5)
plt.plot([y.min(), y.max()], [y.min(), y.max()], "r--")
plt.xlabel("Predicho")
plt.ylabel("Real")
plt.title("Predicho vs Real")

plt.subplot(1, 3, 2)
plt.scatter(y_pred_best, residuos, alpha=0.5)
plt.axhline(y=0, color="r", linestyle="--")
plt.xlabel("Predicho")
plt.ylabel("Residuo")
plt.title("Residuos")

plt.subplot(1, 3, 3)
plt.hist(residuos, bins=50, edgecolor="black")
plt.xlabel("Error ($)")
plt.title("Distribución de Errores")

plt.tight_layout()
plt.savefig("img/analisis_erroses_clv.png")

```

Entregables

1. Pipeline completo con preprocesamiento y 3 modelos
2. Grid Search con el mejor modelo optimizado
3. Feature importance plot
4. Análisis de errores (predicho vs real, residuos)
5. Modelo final serializado (`modelo\_clv.pkl`)

Criterios de Evaluación

Criterio	Puntos
Feature engineering completo	20
Pipeline con 3+ modelos	20
Optimización de hiperparámetros	20
Análisis de errores	20
Feature importance interpretable	10
Modelo serializado	10

¿Has construido un modelo predictivo de CLV listo para producción!

Capítulo 9: Deep Learning con TensorFlow/Keras

9.1 Introducción a Redes Neuronales

9.1.1 El Perceptrón

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, optimizers, regularizers
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Perceptrón simple: una neurona
modelo_lineal = keras.Sequential([
    layers.Dense(1, input_shape=(X_train.shape[1],))
])

modelo_lineal.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

# Equivalente a regresión lineal, pero con redes
historial = modelo_lineal.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=0
)

print(f"Loss final: {historial.history['loss'][-1]:.4f}")
```

9.1.2 Red Multicapa (MLP)

```
# MLP para predicción de ingresos TechStore
mlp = keras.Sequential([
```

```

layers.Input(shape=(X_train.shape[1],)),
layers.Dense(128, activation="relu"),
layers.Dropout(0.3),
layers.Dense(64, activation="relu"),
layers.Dropout(0.2),
layers.Dense(32, activation="relu"),
layers.Dense(1) # regresión
])

mlp.compile(
    optimizer=optimizers.Adam(learning_rate=0.001),
    loss="mse",
    metrics=["mae", "mse"]
)

mlp.summary()

# Callbacks
callbacks = [
    EarlyStopping(patience=10, restore_best_weights=True),
    ReduceLROnPlateau(factor=0.5, patience=5, min_lr=1e-6),
    ModelCheckpoint("models/mlp_best.keras", save_best_only=True)
]

# Entrenamiento
history = mlp.fit(
    X_train, y_train,
    epochs=100,
    batch_size=64,
    validation_split=0.2,
    callbacks=callbacks,
    verbose=1
)

# Evaluación
loss, mae, mse = mlp.evaluate(X_test, y_test)
print(f"Test MAE: ${mae:.2f}, RMSE: ${np.sqrt(mse):.2f}")

```

9.1.3 Regularización en Redes

```

# Comparación de regularización
def crear_mlp(reg_type="none", dropout_rate=0.0):
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train.shape[1],)))

    for units in [128, 64, 32]:
        if reg_type == "l1":
            model.add(layers.Dense(units, activation="relu",
                                    kernel_regularizer=regularizers.l1(0.001)))
        elif reg_type == "l2":
            model.add(layers.Dense(units, activation="relu",
                                    kernel_regularizer=regularizers.l2(0.001)))

```

```

        else:
            model.add(layers.Dense(units, activation="relu"))

        if dropout_rate > 0:
            model.add(layers.Dropout(dropout_rate))

    model.add(layers.Dense(1))
    model.compile(optimizer="adam", loss="mse", metrics=["mae"])
    return model

configs = [
    ("Sin regul.", crear_mlp("none", 0.0)),
    ("L2", crear_mlp("l2", 0.0)),
    ("Dropout 0.3", crear_mlp("none", 0.3)),
    ("L2 + Dropout", crear_mlp("l2", 0.3))
]

for nombre, modelo in configs:
    h = modelo.fit(X_train, y_train, epochs=50, validation_split=0.2,
                    verbose=0, callbacks=[EarlyStopping(patience=5)])
    val_mae = min(h.history["val_mae"])
    print(f"{nombre:15s} | Val MAE: ${val_mae:.2f}")

```

9.2 Redes Convolucionales (CNN)

```

from tensorflow.keras.layers import Conv1D, MaxPooling1D, GlobalAveragePooling1D

# CNN 1D para datos secuenciales de ventas
def crear_cnn_1d(input_shape):
    model = keras.Sequential([
        layers.Input(shape=input_shape),
        Conv1D(filters=64, kernel_size=3, activation="relu"),
        MaxPooling1D(pool_size=2),
        Conv1D(filters=128, kernel_size=3, activation="relu"),
        MaxPooling1D(pool_size=2),
        Conv1D(filters=64, kernel_size=3, activation="relu"),
        GlobalAveragePooling1D(),
        layers.Dense(64, activation="relu"),
        layers.Dropout(0.3),
        layers.Dense(1)
    ])
    return model

# Reshape datos para CNN 1D: (samples, timesteps, features)
n_timesteps = 30
n_features = len(num_features)
X_cnn = X_train.values.reshape(-1, n_timesteps, n_features // n_timesteps)

cnn = crear_cnn_1d((n_timesteps, X_cnn.shape[2]))
cnn.compile(optimizer="adam", loss="mse", metrics=["mae"])
cnn.summary()

```

9.3 Transfer Learning

```
from tensorflow.keras.applications import ResNet50, VGG16
from tensorflow.keras.preprocessing import image

# No tenemos imágenes, pero el concepto aplica a modelos pre-entrenados
# Ejemplo: usar embeddings pre-entrenados para texto
conceptos_transfer = {
    "Feature Extraction": "Congelar capas base, entrenar solo las nuevas",
    "Fine-tuning": "Descongelar últimas capas y re-entrenar con tasa baja",
    "Adaptive": "Añadir capas de adaptación entre el modelo base y el output"
}

# Aplicación a embeddings de productos
# Simular embeddings pre-entrenados
embedding_dim = 300
productos_embedding = np.random.randn(5000, embedding_dim)

modelo_transfer = keras.Sequential([
    layers.Input(shape=(embedding_dim,)),
    layers.Dense(128, activation="relu", trainable=False), # capa congelada
    layers.Dense(64, activation="relu"),
    layers.Dense(1)
])

modelo_transfer.layers[0].trainable = False # feature extraction
modelo_transfer.compile(optimizer="adam", loss="mse")
```

9.4 Ejercicios

1. **MLP:** Construye y entrena un MLP con 3 capas ocultas para predecir ventas de TechStore. Compara con Random Forest.
2. **Regularización:** Experimenta con L1, L2 y Dropout. ¿Qué combinación da mejor generalización?
3. **Callbacks:** Implementa EarlyStopping, ReduceLROnPlateau y ModelCheckpoint juntos. Muestra las curvas de aprendizaje.
4. **CNN 1D:** Diseña una CNN 1D para clasificar patrones de compra semanales.
5. **Transfer Learning:** Investiga y explica cómo aplicarías transfer learning con un modelo pre-entrenado de NLP a reseñas de TechStore.

Capítulo 10: Procesamiento de Lenguaje Natural

10.1 NLP con Python

10.1.1 Preprocesamiento de Texto

```
import re
import nltk
import pandas as pd
import numpy as np

from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer
from nltk.tokenize import word_tokenize

# Descargar recursos de NLTK
nltk.download("punkt")
nltk.download("stopwords")
nltk.download("wordnet")

stemmer = SnowballStemmer("spanish")
stop_words = set(stopwords.words("spanish"))

def preprocesar_texto(texto):
    if not isinstance(texto, str):
        return ""

    # Minúsculas
    texto = texto.lower()

    # Eliminar URLs
    texto = re.sub(r"http\S+|www\S+|https\S+", "", texto)

    # Eliminar signos de puntuación (mantener letras, números y espacios)
    texto = re.sub(r"^\w\sáéíóúüñ", " ", texto)

    # Eliminar números
    texto = re.sub(r"\d+", "", texto)

    # Tokenizar
```

```

tokens = word_tokenize(texto, language="spanish")

# Eliminar stopwords y palabras cortas
tokens = [t for t in tokens if t not in stop_words and len(t) > 2]

# Stemming
tokens = [stemmer.stem(t) for t in tokens]

return " ".join(tokens)

# Cargar reseñas TechStore
resenas = pd.read_parquet("data/techstore_reviews/")
resenas["texto_limpio"] = resenas["review_text"].apply(preprocesar_texto)

print("Ejemplo de preprocesamiento:")
print(f"Original: {resenas['review_text'].iloc[0][:200]}")
print(f"Limpio: {resenas['texto_limpio'].iloc[0][:200]}")

```

10.1.2 Visualización de Texto

```

from wordcloud import WordCloud
import matplotlib.pyplot as plt

# Nube de palabras para reseñas positivas y negativas
positivas = resenas[resenas["rating"] >= 4]["texto_limpio"]
negativas = resenas[resenas["rating"] <= 2]["texto_limpio"]

fig, axes = plt.subplots(1, 2, figsize=(15, 6))

texto_pos = " ".join(positivas.dropna())
wc_pos = WordCloud(width=800, height=400, background_color="white",
                    colormap="Greens", max_words=100).generate(texto_pos)
axes[0].imshow(wc_pos, interpolation="bilinear")
axes[0].axis("off")
axes[0].set_title("Reseñas Positivas (4-5 estrellas)")

texto_neg = " ".join(negativas.dropna())
wc_neg = WordCloud(width=800, height=400, background_color="white",
                    colormap="Reds", max_words=100).generate(texto_neg)
axes[1].imshow(wc_neg, interpolation="bilinear")
axes[1].axis("off")
axes[1].set_title("Reseñas Negativas (1-2 estrellas)")

plt.savefig("img/wordcloud_reviews.png")

```

10.2 Vectorización de Texto

10.2.1 Bag of Words y TF-IDF

```

from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import train_test_split

```

```

from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report

# Bag of Words
bow = CountVectorizer(max_features=1000, ngram_range=(1, 2))
X_bow = bow.fit_transform(resenas["texto_limpio"])

# TF-IDF
tfidf = TfidfVectorizer(max_features=2000, ngram_range=(1, 3), min_df=5)
X_tfidf = tfidf.fit_transform(resenas["texto_limpio"])

print(f"BoW shape: {X_bow.shape}")
print(f"TF-IDF shape: {X_tfidf.shape}")
print(f"Vocabulario TF-IDF: {len(tfidf.get_feature_names_out()):,} términos")

# Clasificación de sentimiento
y_rating = (resenas["rating"] >= 4).astype(int) # 1=positivo, 0=negativo

X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf, y_rating, test_size=0.2, random_state=42
)

nb = MultinomialNB()
nb.fit(X_train, y_train)
y_pred = nb.predict(X_test)

print("\nClasificación de Sentimiento (TF-IDF + Naive Bayes):")
print(classification_report(y_test, y_pred))

# Palabras más informativas para cada clase
log_prob = nb.feature_log_prob_
palabras_pos = tfidf.get_feature_names_out()[np.argsort(log_prob[1])[-20:]]
palabras_neg = tfidf.get_feature_names_out()[np.argsort(log_prob[0])[-20:]]

print(f"\nTop palabras positivas: {' '.join(palabras_pos[::-1])}")
print(f"Top palabras negativas: {' '.join(palabras_neg[::-1])}")

```

10.2.2 Word Embeddings

```

from gensim.models import Word2Vec
from sklearn.decomposition import PCA

# Tokenizar reseñas
tokens = [texto.split() for texto in resenas["texto_limpio"].dropna()]

# Entrenar Word2Vec
w2v = Word2Vec(
    sentences=tokens,
    vector_size=100,
    window=5,
    min_count=5,
    workers=4,

```

```

        epochs=30
    )

    # Palabras similares
    print("Palabras similares a 'producto':")
    for palabra, sim in w2v.wv.most_similar("producto", topn=10):
        print(f"  {palabra}: {sim:.3f}")

    # Embeddings promedio para clasificación
    def document_vector(doc_tokens):
        vectors = [w2v.wv[word] for word in doc_tokens if word in w2v.wv]
        return np.mean(vectors, axis=0) if vectors else np.zeros(w2v.vector_size)

    X_w2v = np.array([document_vector(t) for t in tokens])

    print(f"\nEmbeddings shape: {X_w2v.shape}")

    # Clasificar con embeddings
    X_train_w2v, X_test_w2v = train_test_split(
        range(len(X_w2v)), test_size=0.2, random_state=42
    )

```

10.3 Análisis de Sentimiento con Deep Learning

```

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import Embedding, LSTM, Bidirectional, GlobalMaxPooling1D

# Tokenización Keras
tokenizer = Tokenizer(num_words=5000, oov_token="<OOV>")
tokenizer.fit_on_texts(resenas["texto_limpio"])

sequences = tokenizer.texts_to_sequences(resenas["texto_limpio"])
X_seq = pad_sequences(sequences, maxlen=100, padding="post", truncating="post")

print(f"Secuencias shape: {X_seq.shape}")

# LSTM para clasificación
modelo_lstm = keras.Sequential([
    Embedding(5000, 128, input_length=100),
    Bidirectional(LSTM(64, return_sequences=True)),
    GlobalMaxPooling1D(),
    layers.Dense(32, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid")
])

modelo_lstm.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

```

```

)

modelo_lstm.summary()

# Entrenar
X_seq_train, X_seq_test, y_seq_train, y_seq_test = train_test_split(
    X_seq, y_rating, test_size=0.2, random_state=42
)

hist_lstm = modelo_lstm.fit(
    X_seq_train, y_seq_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2,
    callbacks=[EarlyStopping(patience=3)]
)

# Evaluar
loss, acc = modelo_lstm.evaluate(X_seq_test, y_seq_test)
print(f"Test Accuracy (LSTM): {acc:.3f}")

```

10.4 Ejercicios

1. **Preprocesamiento:** Crea una función de preprocesamiento mejorada que maneje emojis, hashtags y menciones.
2. **TF-IDF:** Clasifica reseñas en 3 niveles (positivo, neutral, negativo) usando TF-IDF + Logistic Regression.
3. **Word2Vec:** Entrena Word2Vec con las reseñas y encuentra las 10 palabras más similares a "envío", "calidad" y "precio".
4. **LSTM:** Compara el rendimiento de LSTM unidireccional vs bidireccional para análisis de sentimiento.
5. **Visualización:** Genera wordclouds separadas para cada categoría de producto y analiza las diferencias.

Capítulo 11: Series Temporales y Forecasting

11.1 Introducción a Series Temporales

11.1.1 Componentes de una Serie Temporal

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller, acf, pacf
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Datos de ventas diarias TechStore
ventas_diarias = pd.read_parquet("data/silver/ventas_diarias/")
ventas_diarias = ventas_diarias.set_index("fecha").sort_index()

print(f"Rango: {ventas_diarias.index.min()} → {ventas_diarias.index.max()}")
print(f"Registros: {len(ventas_diarias):,}")
print(f"Frecuencia: {pd.infer_freq(ventas_diarias.index)}")

# Visualización
plt.figure(figsize=(15, 8))
plt.plot(ventas_diarias["ingresos"], label="Ingresos diarios")
plt.title("Ventas Diarias TechStore 2025")
plt.xlabel("Fecha")
plt.ylabel("Ingresos ($)")
plt.legend()
plt.savefig("img/serie_temporal_ventas.png")

# Descomposición
descomposicion = seasonal_decompose(
    ventas_diarias["ingresos"],
    model="additive",
    period=7 # estacionalidad semanal
)

fig, axes = plt.subplots(4, 1, figsize=(15, 10))
descomposicion.observed.plot(ax=axes[0], title="Original")
descomposicion.trend.plot(ax=axes[1], title="Tendencia")
```

```

descomposicion.seasonal.plot(ax=axes[2], title="Estacionalidad")
descomposicion.resid.plot(ax=axes[3], title="Residuo")
plt.tight_layout()
plt.savefig("img/descomposicion_serie.png")

```

11.1.2 Estacionariedad

```

# Test de Dickey-Fuller
def test_estacionariedad(serie, nombre=""):
    resultado = adfuller(serie.dropna())
    print(f"=== {nombre} ===")
    print(f"ADF Statistic: {resultado[0]:.4f}")
    print(f"p-value: {resultado[1]:.6f}")
    print(f"Valores críticos:")
    for key, value in resultado[4].items():
        print(f"  {key}: {value:.4f}")

    if resultado[1] <= 0.05:
        print(" Serie ESTACIONARIA (rechazamos H0)")
    else:
        print(" Serie NO estacionaria (no rechazamos H0)")

test_estacionariedad(ventas_diarias["ingresos"], "Ingresos Original")

# Diferenciación
ventas_diarias["ingresos_diff1"] = ventas_diarias["ingresos"].diff()
test_estacionariedad(ventas_diarias["ingresos_diff1"], "Ingresos Diff(1)")

# ACF y PACF
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
plot_acf(ventas_diarias["ingresos_diff1"].dropna(), ax=axes[0], lags=40)
plot_pacf(ventas_diarias["ingresos_diff1"].dropna(), ax=axes[1], lags=40)
plt.savefig("img/acf_pacf.png")

```

11.2 ARIMA

```

from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error
import warnings
warnings.filterwarnings("ignore")

# Train/Test split temporal
train = ventas_diarias["ingresos"][:-30]
test = ventas_diarias["ingresos"][-30:]

# ARIMA(p,d,q)
modelo_arima = ARIMA(train, order=(7, 1, 7))
modelo_arima_fit = modelo_arima.fit()

print(modelo_arima_fit.summary())

```

```

# Predicción
pred_arima = modelo_arima_fit.forecast(steps=30)

# Evaluación
rmse_arima = np.sqrt(mean_squared_error(test, pred_arima))
mae_arima = mean_absolute_error(test, pred_arima)
print(f"ARIMA(7,1,7) - RMSE: ${rmse_arima:.2f}, MAE: ${mae_arima:.2f}")

# Visualización
plt.figure(figsize=(12, 6))
plt.plot(train.index[-60:], train[-60:], label="Train")
plt.plot(test.index, test, label="Test real")
plt.plot(test.index, pred_arima, label="ARIMA Pred", linestyle="--")
plt.legend()
plt.title("Forecasting con ARIMA")
plt.savefig("img/forecast_arima.png")

```

11.3 Prophet (Meta)

```

from prophet import Prophet

# Preparar datos para Prophet
df_prophet = ventas_diarias.reset_index()[["fecha", "ingresos"]]
df_prophet.columns = ["ds", "y"]

# Crear y entrenar modelo
modelo_prophet = Prophet(
    yearly_seasonality=False,
    weekly_seasonality=True,
    daily_seasonality=False,
    seasonality_mode="multiplicative",
    changepoint_prior_scale=0.05
)

# Añadir regresores externos
df_prophet["es_fin_semana"] = (df_prophet["ds"].dt.dayofweek >= 5).astype(int)
df_prophet["dia_mes"] = df_prophet["ds"].dt.day

modelo_prophet.add_regressor("es_fin_semana")
modelo_prophet.add_regressor("dia_mes")

modelo_prophet.fit(df_prophet)

# Predecir 30 días
future = modelo_prophet.make_future_dataframe(periods=30)
future["es_fin_semana"] = (future["ds"].dt.dayofweek >= 5).astype(int)
future["dia_mes"] = future["ds"].dt.day

forecast = modelo_prophet.predict(future)

# Visualización

```

```

fig = modelo_prophet.plot(forecast)
plt.title("Forecasting con Prophet")
plt.savefig("img/forecast_prophet.png")

# Componentes
fig2 = modelo_prophet.plot_components(forecast)
plt.savefig("img/prophet_components.png")

# Evaluación
pred_prophet = forecast.set_index("ds").loc[test.index, "yhat"]
rmse_prophet = np.sqrt(mean_squared_error(test, pred_prophet))
mae_prophet = mean_absolute_error(test, pred_prophet)
print(f"Prophet - RMSE: ${rmse_prophet:.2f}, MAE: ${mae_prophet:.2f}")

```

11.4 LSTM para Series Temporales

```

from sklearn.preprocessing import MinMaxScaler

# Preparar datos para LSTM
scaler = MinMaxScaler()
serie_scaled = scaler.fit_transform(ventas_diarias[["ingresos"]])

def crear_secuencias(data, seq_length=14):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i+seq_length])
        y.append(data[i+seq_length])
    return np.array(X), np.array(y)

seq_length = 14
X_seq_ts, y_seq_ts = crear_secuencias(serie_scaled, seq_length)

# Split temporal
split = len(X_seq_ts) - 30
X_train_ts, X_test_ts = X_seq_ts[:split], X_seq_ts[split:]
y_train_ts, y_test_ts = y_seq_ts[:split], y_seq_ts[split:]

# Modelo LSTM
modelo_lstm_ts = keras.Sequential([
    layers.LSTM(50, return_sequences=True, input_shape=(seq_length, 1)),
    layers.Dropout(0.2),
    layers.LSTM(50, return_sequences=False),
    layers.Dropout(0.2),
    layers.Dense(1)
])

modelo_lstm_ts.compile(optimizer="adam", loss="mse", metrics=["mae"])
modelo_lstm_ts.summary()

# Entrenar
hist_lstm_ts = modelo_lstm_ts.fit(

```

```

X_train_ts, y_train_ts,
epochs=50,
batch_size=16,
validation_split=0.1,
callbacks=[EarlyStopping(patience=5)],
verbose=1
)

# Predecir
pred_scaled = modelo_lstm_ts.predict(X_test_ts)
pred_lstm = scaler.inverse_transform(pred_scaled)
real_lstm = scaler.inverse_transform(y_test_ts)

rmse_lstm = np.sqrt(mean_squared_error(real_lstm, pred_lstm))
mae_lstm = mean_absolute_error(real_lstm, pred_lstm)
print(f"LSTM - RMSE: ${rmse_lstm:.2f}, MAE: ${mae_lstm:.2f}")

# Comparación final
resultados_ts = pd.DataFrame({
    "Modelo": ["ARIMA(7,1,7)", "Prophet", "LSTM(14 días)"],
    "RMSE": [rmse_arima, rmse_prophet, rmse_lstm],
    "MAE": [mae_arima, mae_prophet, mae_lstm]
})
print("\n=== Comparación de Modelos ===")
print(resultados_ts.to_string(index=False))

```

11.5 Ejercicios

1. **Descomposición:** Descompón las ventas semanales de TechStore en tendencia, estacionalidad y residuo. ¿Qué patrones observas?
2. **ARIMA:** Encuentra los mejores parámetros (p,d,q) usando auto_arima o análisis de ACF/PACF.
3. **Prophet:** Añade 3 regresores externos (día festivo, promociones, temperatura) y mide el impacto en el forecast.
4. **LSTM:** Experimenta con diferentes sequence_length (7, 14, 30) y número de unidades LSTM.
5. **Comparación:** Evalúa ARIMA, Prophet y LSTM en los mismos 30 días de test y determina el mejor modelo para el negocio.

Capítulo 12: MLOps: Machine Learning en Producción

12.1 Introducción a MLOps

MLOps es el conjunto de prácticas que unen el desarrollo de ML con las operaciones de producción. Es la respuesta a: "funciona en mi Jupyter, pero ¿cómo lo llevo a producción?"

```
pilares_mlops = {
    "Experimentación": "Seguimiento de parámetros, métricas, artefactos",
    "Reproducibilidad": "Versionado de datos, código, modelos, entorno",
    "Despliegue": "APIs, batch inference, edge deployment",
    "Monitoreo": "Data drift, model decay, alertas automáticas",
    "Gobernanza": "Auditoría, explicabilidad, fairness, compliance"
}
```

12.2 MLflow: Experiment Tracking

12.2.1 Configuración Inicial

```
import mlflow
import mlflow.sklearn
from mlflow.tracking import MlflowClient

# Configurar MLflow
mlflow.set_tracking_uri("http://localhost:5000") # o "file:./mlruns"
mlflow.set_experiment("techstore-clv-prediction")

# O usar tracking local
# mlflow.set_tracking_uri("./mlruns")
```

12.2.2 Registro de Experimentos

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import numpy as np

# Experimentar con diferentes modelos
modelos_exp = {
```

```

"RandomForest": {
    "modelo": RandomForestRegressor(n_estimators=200, random_state=42),
    "params": {"n_estimators": 200, "max_depth": "default"}
},
"XGBoost": {
    "modelo": XGBRegressor(n_estimators=200, learning_rate=0.05, random_state=42),
    "params": {"n_estimators": 200, "learning_rate": 0.05, "max_depth": 6}
},
"GradientBoosting": {
    "modelo": GradientBoostingRegressor(n_estimators=100, random_state=42),
    "params": {"n_estimators": 100, "learning_rate": 0.1}
}
}

for nombre, config in modelos_exp.items():
    with mlflow.start_run(run_name=nombre):
        # Log parámetros
        for key, value in config["params"].items():
            mlflow.log_param(key, value)

        # Entrenar
        pipeline = Pipeline([
            ("preprocessor", preprocessor),
            ("regressor", config["modelo"])
        ])
        pipeline.fit(X_train, y_train)

        # Predecir
        y_pred = pipeline.predict(X_test)

        # Log métricas
        mlflow.log_metric("rmse", np.sqrt(mean_squared_error(y_test, y_pred)))
        mlflow.log_metric("mae", mean_absolute_error(y_test, y_pred))
        mlflow.log_metric("r2", r2_score(y_test, y_pred))

        # Log modelo
        mlflow.sklearn.log_model(pipeline, f"model_{nombre}")

        # Log artefactos adicionales
        mlflow.log_artifact("img/feature_importance_clv.png")
        mlflow.log_artifact("img/analisisErrores_clv.png")

        print(f"Run {nombre} completado: R2={r2_score(y_test, y_pred):.3f}")

```

12.2.3 Comparación de Runs

```

# Cargar experimentos
client = MlflowClient()
experiment = client.get_experiment_by_name("techstore-clv-prediction")

runs = client.search_runs(
    experiment_ids=[experiment.experiment_id],

```

```

        order_by=["metrics.r2 DESC"],
        max_results=5
    )

    print("=== Top 5 Runs ===")
    for run in runs:
        print(f" {run.data.tags.get('mlflow.runName', 'unknown')}:20s} "
              f"| R2: {run.data.metrics.get('r2', 0):.3f} "
              f"| RMSE: ${run.data.metrics.get('rmse', 0):.2f}")

    # Obtener mejor modelo
    best_run = runs[0]
    best_model_uri = f"runs:{best_run.info.run_id}/model_XGBoost"
    best_model = mlflow.sklearn.load_model(best_model_uri)

    print(f"\nMejor modelo cargado desde: {best_model_uri}")

```

12.2.4 Model Registry

```

# Registrar modelo en MLflow Model Registry
result = mlflow.register_model(
    model_uri=best_model_uri,
    name="TechStore_CLV_Model"
)

# Transicionar a diferentes stages
client.transition_model_version_stage(
    name="TechStore_CLV_Model",
    version=result.version,
    stage="Staging"
)

# Promover a producción
client.transition_model_version_stage(
    name="TechStore_CLV_Model",
    version=result.version,
    stage="Production"
)

# Listar versiones
for mv in client.search_model_versions("name='TechStore_CLV_Model'"):
    print(f"Versión {mv.version}: stage={mv.current_stage}, "
          f"run_id={mv.run_id[:8]}...")

```

12.3 Versionado de Modelos

```

# Versionado manual con DVC (Data Version Control)
# dvc init
# dvc add data/
# dvc add models/

```

```

# Versionado automático con MLflow
import joblib
from datetime import datetime

def guardar_modelo_versionado(modelo, metricas, nombre="modelo_clv"):
    version = datetime.now().strftime("%Y%m%d_%H%M%S")
    path = f"models/{nombre}_{version}.pkl"

    # Guardar modelo
    joblib.dump(modelo, path)

    # Guardar metadata
    metadata = {
        "version": version,
        "fecha": datetime.now().isoformat(),
        "metricas": metricas,
        "features": X_train.columns.tolist()
    }
    joblib.dump(metadata, f"models/{nombre}_{version}_metadata.pkl")

    print(f"Modelo guardado: {path}")
    return version

# Ejemplo de uso
metricas = {"rmse": rmse, "r2": r2, "mae": mae}
version = guardar_modelo_versionado(best_model, metricas)

# Cargar modelo específico
def cargar_modelo_version(version):
    return joblib.load(f"models/modelo_clv_{version}.pkl")

```

12.4 Monitoreo de Modelos en Producción

12.4.1 Data Drift Detection

```

from scipy.stats import ks_2samp
import warnings
warnings.filterwarnings("ignore")

def detectar_data_drift(datos_referencia, datos_nuevos, features, threshold=0.05):
    """
    Detectar drift entre datos de entrenamiento y nuevos datos
    usando Kolmogorov-Smirnov test
    """
    drifts = []
    for feature in features:
        stat, p_value = ks_2samp(datos_referencia[feature], datos_nuevos[feature])
        if p_value < threshold:
            drifts.append({
                "feature": feature,
                "statistic": stat,
                "p_value": p_value,
            })

```

```

        "drift": True
    })

    return pd.DataFrame(drifts)

# Simular datos nuevos con drift
datos_nuevos = X_test.copy()
datos_nuevos["freq_total"] = datos_nuevos["freq_total"] * 1.5 # drift artificial

drift_result = detectar_data_drift(
    X_train, datos_nuevos,
    features=num_features
)

print(f"Features con drift detectado: {len(drift_result)}")
if len(drift_result) > 0:
    print(drift_result[["feature", "p_value"]].to_string(index=False))

```

12.4.2 Performance Monitoring

```

import logging
from datetime import datetime, timedelta

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("model_monitor")

# Configuración de alertas
config_monitoreo = {
    "rmse_threshold": 50.0,
    "mae_threshold": 35.0,
    "min_predicciones": 100,
    "ventana_drift": "7d",
    "alertas_email": ["data-team@techstore.com"]
}

def evaluar_modelo_produccion():
    """Evaluar rendimiento del modelo en producción"""
    ventana = datetime.now() - timedelta(days=7)
    datos_recientes = obtener_datos_desde(ventana)

    if len(datos_recientes) < config_monitoreo["min_predicciones"]:
        logger.warning(f"Datos insuficientes: {len(datos_recientes)}")
        return

    # Predecir y evaluar
    X_reciente = datos_recientes.drop(["customer_id", "monto_total"], axis=1)
    y_reciente = datos_recientes["monto_total"]
    y_pred = best_model.predict(X_reciente)

    rmse_actual = np.sqrt(mean_squared_error(y_reciente, y_pred))
    mae_actual = mean_absolute_error(y_reciente, y_pred)

```

```

logger.info(f"RMSE actual: ${rmse_actual:.2f}")
logger.info(f"MAE actual: ${mae_actual:.2f}")

# Alertas
if rmse_actual > config_monitoreo["rmse_threshold"]:
    logger.error(f" RMSE excedió umbral: ${rmse_actual:.2f}")
    enviar_alerta(f"RMSE crítico: ${rmse_actual:.2f}")

return {"rmse": rmse_actual, "mae": mae_actual, "fecha": datetime.now()}

```

12.5 Ejercicios

1. **MLflow Setup:** Configura MLflow (local o cloud) y registra 3 experimentos con diferentes modelos de TechStore.
2. **Model Registry:** Promueve un modelo de Staging a Production y demuestra cómo cargar cada versión.
3. **Data Drift:** Implementa detección de drift para las features numéricas del modelo CLV. Simula 3 escenarios de drift.
4. **Dashboard de Monitoreo:** Crea un dashboard en Streamlit que muestre métricas en tiempo real del modelo en producción.
5. **Pipeline MLOps:** Diseña un pipeline completo: entrenamiento registro staging production monitoreo alertas.

Checkpoint 3: Pipeline de ML Completo

Objetivo

Construir un pipeline de machine learning completo que integre experimentación (MLflow), forecasting (Prophet/LSTM), NLP (análisis de sentimiento de reseñas) y MLOps con monitoreo de drift.

Dataset

- `techstore\_transactions.parquet` — para modelo de ventas
- `techstore\_reviews.parquet` — para NLP
- `techstore\_customers.parquet` — para features

Pipeline

Fase 1: Experimentación con MLflow

```
import mlflow
import mlflow.sklearn
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, VotingRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import joblib

mlflow.set_experiment("techstore-ml-pipeline")

df = pd.read_parquet("data/silver/ventas/")
y = df["amount"]
X = df.drop(["amount", "transaction_id", "timestamp"], axis=1)

num_features = X.select_dtypes(include=[np.number]).columns.tolist()
cat_features = X.select_dtypes(include=["object", "category"]).columns.tolist()

preprocessor = ColumnTransformer([
```

```

        ("num", StandardScaler(), num_features),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_features)
    ])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

modelos = {
    "RF": RandomForestRegressor(n_estimators=200, random_state=42),
    "GB": GradientBoostingRegressor(n_estimators=100, random_state=42),
    "XGB": XGBRegressor(n_estimators=200, learning_rate=0.05, random_state=42),
    "Ensemble": VotingRegressor([
        ("rf", RandomForestRegressor(n_estimators=100, random_state=42)),
        ("xgb", XGBRegressor(n_estimators=100, learning_rate=0.05, random_state=42))
    ])
}

mejor_modelo = None
mejor_r2 = -np.inf

for nombre, modelo in modelos.items():
    with mlflow.start_run(run_name=nombre):
        pipeline = Pipeline([("preprocessor", preprocessor), ("model", modelo)])
        cv_scores = cross_val_score(pipeline, X_train, y_train, cv=3, scoring="r2")

        pipeline.fit(X_train, y_train)
        y_pred = pipeline.predict(X_test)

        r2 = r2_score(y_test, y_pred)
        rmse = np.sqrt(mean_squared_error(y_test, y_pred))
        mae = mean_absolute_error(y_test, y_pred)

        mlflow.log_param("model_type", nombre)
        mlflow.log_metric("cv_r2_mean", cv_scores.mean())
        mlflow.log_metric("test_r2", r2)
        mlflow.log_metric("test_rmse", rmse)
        mlflow.log_metric("test_mae", mae)
        mlflow.sklearn.log_model(pipeline, f"model_{nombre}")

        print(f"{nombre}: R2={r2:.3f}, RMSE=${rmse:.2f}, CV={cv_scores.mean():.3f}")

        if r2 > mejor_r2:
            mejor_r2 = r2
            mejor_modelo = pipeline

# Registrar mejor modelo
mlflow.register_model(
    f"runs:/{mlflow.active_run().info.run_id}/model_XGB",
    "TechStore_Predictor"
)

```

Fase 2: Forecast de Ventas (Prophet + LSTM)

```

from prophet import Prophet
from tensorflow import keras
from tensorflow.keras import layers

# Prophet
ventas_diarias = pd.read_parquet("data/silver/ventas_diarias/")
df_prophet = ventas_diarias.reset_index()[["fecha", "ingresos"]]
df_prophet.columns = ["ds", "y"]

modelo_prophet = Prophet(weekly_seasonality=True, seasonality_mode="multiplicative")
modelo_prophet.fit(df_prophet)
future = modelo_prophet.make_future_dataframe(periods=14)
forecast = modelo_prophet.predict(future)

# Guardar forecast
forecast[["ds", "yhat", "yhat_lower", "yhat_upper"]].to_csv("data/gold/forecast_14d.csv", index=False)
print("Forecast generado: data/gold/forecast_14d.csv")

```

Fase 3: Análisis de Sentimiento de Reseñas

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline as SkPipeline
import joblib

resenas = pd.read_parquet("data/techstore_reviews/")
resenas["sentimiento"] = (resenas["rating"] >= 4).astype(int)

pipeline_nlp = SkPipeline([
    ("tfidf", TfidfVectorizer(max_features=2000, ngram_range=(1, 2), min_df=5)),
    ("clf", LogisticRegression(class_weight="balanced"))
])

X_nlp_train, X_nlp_test, y_nlp_train, y_nlp_test = train_test_split(
    resenas["review_text"], resenas["sentimiento"],
    test_size=0.2, random_state=42
)

pipeline_nlp.fit(X_nlp_train, y_nlp_train)
acc = pipeline_nlp.score(X_nlp_test, y_nlp_test)
print(f"Sentiment Accuracy: {acc:.3f}")

joblib.dump(pipeline_nlp, "models/sentiment_model.pkl")

```

Fase 4: Monitoreo MLOps

```

def verificar_salud_modelos():
    from scipy.stats import ks_2samp

    reporte = {
        "fecha": pd.Timestamp.now().isoformat(),
        "modelo_principal": "TechStore_Predictor (XGBoost)",

```

```

        "r2_ultimo_test": mejor_r2,
        "forecast_disponible": True,
        "sentiment_accuracy": acc
    }

    # Detectar drift en datos nuevos simulados
    datos_recientes = df.sample(frac=0.1)
    drift_features = []

    for col in num_features[:5]:
        stat, p = ks_2samp(X_train[col], datos_recientes[col])
        if p < 0.05:
            drift_features.append(col)

    reporte["data_drift"] = len(drift_features) == 0
    reporte["features_con_drift"] = drift_features

    return pd.Series(reporte)

print("=== Reporte de Salud MLOps ===")
print(verificar_salud_modelos())

```

Entregables

1. Pipeline ML con experiment tracking (MLflow)
2. Modelo registrado en Model Registry
3. Forecast de ventas a 14 días
4. Modelo de sentimiento para reseñas
5. Reporte de monitoreo MLOps

Criterios de Evaluación

Criterio	Puntos
Experiment tracking completo	20
Mejor modelo registrado	15
Forecast funcional	15
NLP pipeline funcional	15
Monitoreo y drift detection	15
Integración de todos los componentes	20

¡Has construido un pipeline ML completo que integra experimentación, forecasting y MLOps!

Capítulo 13: LLMs e Inteligencia Artificial Generativa

13.1 Introducción a los LLMs

Los Large Language Models (LLMs) han revolucionado la IA. Modelos como GPT-4, Claude y Llama pueden entender y generar texto con una fluidez sin precedentes.

13.1.1 Arquitectura Transformer

```
# Concepto: Attention is All You Need
conceptos_transformer = {
    "Self-Attention": "Cada palabra mira a todas las demás para entender contexto",
    "Multi-Head Attention": "Múltiples cabezas de atención en paralelo",
    "Positional Encoding": "Codificación de posición de palabras en la secuencia",
    "Feed Forward": "Red densa después de la atención",
    "Layer Norm": "Normalización para estabilidad del entrenamiento"
}
```

13.1.2 Modelos Disponibles

```
modelos_llm = {
    "Propietarios": ["GPT-4 (OpenAI)", "Claude (Anthropic)", "Gemini (Google)"],
    "Open Source": ["Llama 3 (Meta)", "Mistral", "Falcon", "Phi-3 (Microsoft)"],
    "Cloud": ["Vertex AI (Gemini)", "Amazon Bedrock", "Azure OpenAI"]
}
```

13.2 LangChain: Orquestación de LLMs

13.2.1 Configuración

```
from langchain_community.llms import Ollama
from langchain_openai import ChatOpenAI
from langchain_google_vertexai import VertexAI
from langchain.prompts import PromptTemplate, ChatPromptTemplate
from langchain.schema import SystemMessage, HumanMessage, AIMessage
from langchain.chains import LLMChain, RetrievalQA, SimpleSequentialChain
from langchain.memory import ConversationBufferMemory
from langchain.callbacks import StreamingStdOutCallbackHandler

# Opciones de LLM
```

```
llm_local = Ollama(model="llama3", temperature=0.7)
llm_cloud = ChatOpenAI(model="gpt-4", temperature=0.3)
llm_vertex = VertexAI(model="gemini-pro", temperature=0.5)
```

13.2.2 Prompts y Chains

```
# Template básico
template_analista = """
Eres un asistente experto en datos de TechStore, una tienda de tecnología.
Responde la pregunta basándote en el contexto proporcionado.

Contexto: {contexto}

Pregunta: {pregunta}

Respuesta:
"""

prompt = PromptTemplate(
    template=template_analista,
    input_variables=["contexto", "pregunta"]
)

# Chain simple
chain = LLMChain(
    llm=llm_cloud,
    prompt=prompt,
    verbose=True
)

# Usar
respuesta = chain.invoke({
    "contexto": "TechStore vendió 10M en electrónicos en Q1 2025",
    "pregunta": "¿Cuánto vendió TechStore en electrónicos?"
})
print(respuesta["text"])
```

13.2.3 Memoria y Conversación

```
from langchain.chains import ConversationChain

# Memoria conversacional
memory = ConversationBufferMemory()
conversacion = ConversationChain(
    llm=llm_cloud,
    memory=memory,
    verbose=True
)

# Chat continuo
resp1 = conversacion.predict(input="¿Cuáles son las categorías de TechStore?")
print(resp1)
```

```
resp2 = conversacion.predict(input="¿Cuál fue la más vendida en 2025?")
print(resp2)
# La memoria recuerda que hablamos de TechStore
```

13.3 RAG: Retrieval-Augmented Generation

RAG combina recuperación de información con generación de texto. Es la técnica clave para hacer que los LLMs trabajen con tus datos.

13.3.1 Vector Store

```
from langchain_community.vectorstores import FAISS, Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.document_loaders import DataFrameLoader, TextLoader, DirectoryLoader

# Cargar documentos TechStore
loader = DirectoryLoader(
    "data/documentacion/",
    glob="**/*.md",
    loader_cls=TextLoader
)
documentos = loader.load()

print(f"Documentos cargados: {len(documentos)}")

# Dividir en chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=["\n\n", "\n", ".", "!", "?", ",", " ", " ", " "]
)
chunks = text_splitter.split_documents(documentos)
print(f"Chunks creados: {len(chunks)}")

# Embeddings
embeddings = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"
)

# Vector store
vectorstore = FAISS.from_documents(chunks, embeddings)
vectorstore.save_local("data/vectorstore/techstore_docs")
```

13.3.2 RetrievalQA Chain

```
from langchain.chains import RetrievalQA

# Crear retrievers
retriever = vectorstore.as_retriever(
    search_type="similarity",
```

```

        search_kwargs={"k": 5}
    )

    # Chain QA con RAG
    qa_chain = RetrievalQA.from_chain_type(
        llm=llm_cloud,
        chain_type="stuff", # 'stuff', 'map_reduce', 'refine', 'map_rerank'
        retriever=retriever,
        return_source_documents=True,
        verbose=True
    )

    # Preguntas sobre TechStore
    preguntas = [
        "¿Cómo se calcula el CLV en TechStore?",
        "¿Qué productos tienen mayor rotación?",
        "¿Cuál es el perfil del cliente premium?"
    ]

    for pregunta in preguntas:
        print(f"\nPregunta: {pregunta}")
        resultado = qa_chain.invoke({"query": pregunta})
        print(f"Respuesta: {resultado['result'][:200]}...")
        print(f"Fuentes: {len(resultado['source_documents'])} documentos")

```

13.3.3 RAG sobre TechStore Reviews

```

# Crear RAG específico para reseñas
loader_reviews = DataFrameLoader(resenas, page_content_column="review_text")
docs_reviews = loader_reviews.load()
chunks_reviews = text_splitter.split_documents(docs_reviews)

vectorstore_reviews = FAISS.from_documents(chunks_reviews, embeddings)

qa_reviews = RetrievalQA.from_chain_type(
    llm=llm_cloud,
    chain_type="stuff",
    retriever=vectorstore_reviews.as_retriever(k=3)
)

# Consultas sobre reseñas
consultas = [
    "¿Qué dicen los clientes sobre el envío?",
    "¿Cuáles son las quejas más comunes sobre laptops?",
    "¿Qué productos tienen mejores reseñas?"
]

for consulta in consultas:
    print(f"\n--- {consulta} ---")
    print(qa_reviews.invoke({"query": consulta})["result"][:300])

```

13.4 Agentes con LangChain

```
from langchain.agents import AgentExecutor, create_react_agent, Tool
from langchain.tools import tool
from langchain_community.tools import DuckDuckGoSearchRun

# Herramientas personalizadas
@tool
def consultar_ventas(query: str) -> str:
    """Consulta métricas de ventas de TechStore"""
    df = pd.read_parquet("data/gold/ventas_diarias/")
    return f"Ventas totales: ${df['ingresos'].sum():,.2f}"

@tool
def buscar_producto(query: str) -> str:
    """Busca información de productos en TechStore"""
    df = pd.read_parquet("data/techstore_products/")
    resultado = df[df["product_name"].str.contains(query, case=False, na=False)]
    return resultado.to_string()

# Crear agente
tools = [consultar_ventas, buscar_producto]

prompt_agente = """
Eres un asistente de datos de TechStore. Tienes acceso a las siguientes herramientas:
{tools}

Usa las herramientas para responder preguntas sobre ventas y productos.

Pregunta: {input}

{agent_scratchpad}
"""

agente = create_react_agent(
    llm=llm_cloud,
    tools=tools,
    prompt=PromptTemplate.from_template(prompt_agente)
)

agent_executor = AgentExecutor(
    agent=agente,
    tools=tools,
    verbose=True,
    handle_parsing_errors=True
)

# Probar agente
respuesta = agent_executor.invoke({
    "input": "¿Cuánto vendió TechStore en total y qué productos son laptops?"
})
print(respuesta["output"])
```

13.5 Ejercicios

1. **Prompts:** Diseña 3 prompts diferentes para que un LLM analice datos de ventas de TechStore. Compara la calidad de las respuestas.
2. **RAG:** Construye un sistema RAG completo con documentación técnica de TechStore y prueba 5 preguntas de negocio.
3. **Vector Store:** Compara FAISS vs Chroma como vector store. ¿Cuál es más rápido? ¿Cuál escala mejor?
4. **Agentes:** Crea un agente con 3 herramientas personalizadas que pueda responder preguntas complejas sobre TechStore.
5. **Evaluación:** Diseña un conjunto de evaluación para medir la calidad de un sistema RAG (faithfulness, relevance, helpfulness).

Capítulo 14: Machine Learning en la Nube

14.1 BigQuery ML

BigQuery ML permite crear y ejecutar modelos de ML directamente con SQL, sin mover datos.

14.1.1 Configuración

```
from google.cloud import bigquery
from google.cloud import aiplatform
import pandas as pd

# Cliente BigQuery
client = bigquery.Client(project="techstore-analytics")

# Verificar datasets
datasets = list(client.list_datasets())
print("Datasets disponibles:")
for ds in datasets:
    print(f" {ds.dataset_id}")
```

14.1.2 Modelos con SQL

```
-- Modelo de regresión lineal para predecir CLV
CREATE OR REPLACE MODEL `techstore.clv_model`
OPTIONS(
  model_type='linear_reg',
  input_label_cols=['total_spent'],
  l1_reg=0.01,
  l2_reg=0.01,
  max_iterations=50
) AS
SELECT
  customer_id,
  visit_count,
  avg_session_duration,
  categories_viewed,
  days_since_last_visit,
  total_spent
FROM `techstore.features.train_data`
```

```

# Evaluar modelo
query_eval = """
SELECT
    *
FROM ML.EVALUATE(MODEL `techstore.clv_model`,
    TABLE `techstore.features.eval_data`)
"""

eval_result = client.query(query_eval).to_dataframe()
print(eval_result)

# Predecir
query_pred = """
SELECT
    customer_id,
    predicted_total_spent AS predicted_clv
FROM ML.PREDICT(MODEL `techstore.clv_model`,
    TABLE `techstore.features.new_customers`)
ORDER BY predicted_clv DESC
LIMIT 10
"""

predicciones = client.query(query_pred).to_dataframe()
print(predicciones)

```

14.1.3 XGBoost en BigQuery

```

-- Modelo XGBoost para clasificación de clientes premium
CREATE OR REPLACE MODEL `techstore.premium_classifier`
OPTIONS(
    model_type='boosted_tree_classifier',
    input_label_cols=['is_premium'],
    l1_reg=0.01,
    l2_reg=0.01,
    num_parallel_tree=8,
    max_tree_depth=6,
    subsample=0.8,
    learn_rate=0.1,
    min_tree_child_weight=5,
    early_stop=True
) AS
SELECT
    visit_count,
    avg_session_duration,
    categories_viewed,
    products_viewed,
    days_since_last_visit,
    is_premium
FROM `techstore.features.train_data`

```

```

# Feature importance
query_importance = """
SELECT
    input,

```

```

        weight
    FROM ML.FEATURE_IMPORTANCE(MODEL `techstore.premium_classifier`)
    ORDER BY weight DESC
    """

    importance = client.query(query_importance).to_dataframe()
    print("Feature Importance (BigQuery ML):")
    print(importance)

```

14.2 Vertex AI

14.2.1 Custom Training

```

from google.cloud.aiplatform import CustomTrainingJob
from google.cloud.aiplatform import hyperparameter_tuning as hp

# Configurar Vertex AI
aiplatform.init(
    project="techstore-analytics",
    location="us-central1",
    staging_bucket="gs://techstore-ml-staging"
)

# Job de entrenamiento personalizado
job = CustomTrainingJob(
    display_name="techstore-xgboost-training",
    script_path="trainers/train_xgboost.py",
    container_uri="us-docker.pkg.dev/vertex-ai/training/xgboost-cpu.1-1:latest",
    requirements=["pandas", "scikit-learn", "cloudml-hypertune"],
    model_serving_container_image_uri="us-docker.pkg.dev/vertex-ai/prediction/xgboost-cpu.1-1:latest"
)

# Ejecutar
model = job.run(
    model_display_name="techstore-clv-predictor",
    replica_count=1,
    machine_type="n1-standard-4",
    args=["--data-dir=gs://techstore-ml-staging/data"]
)

```

14.2.2 AutoML

```

from google.cloud.aiplatform import AutoMLTabularTrainingJob

# AutoML para clasificación
automl_job = AutoMLTabularTrainingJob(
    display_name="techstore-automl-premium",
    optimization_prediction_type="classification",
    optimization_objective="maximize-au-roc"
)

# Dataset

```

```

dataset = aiplatform.TabularDataset.create(
    display_name="techstore-customers",
    gcs_source=["gs://techstore-ml-staging/data/customers_for_automl.csv"]
)

# Entrenar
model_automl = automl_job.run(
    dataset=dataset,
    target_column="is_premium",
    budget_milli_node_hours=1000,
    model_display_name="techstore-automl-premium-model"
)

# Evaluar
evaluation = model_automl.evaluate()
print(f"AutoML AUC-ROC: {evaluation.metrics['auRoc']:.3f}")

```

14.2.3 Endpoint y Predicciones

```

# Desplegar modelo
endpoint = model.deploy(
    machine_type="n1-standard-2",
    min_replica_count=1,
    max_replica_count=3,
    traffic_split={"0": 100}
)

# Predicción batch
batch_prediction_job = model.batch_predict(
    job_display_name="techstore-batch-prediction",
    gcs_source="gs://techstore-ml-staging/data/to_predict.csv",
    gcs_destination_prefix="gs://techstore-ml-staging/predictions/",
    machine_type="n1-standard-4"
)

# Predicción online
instances = [
    {"visit_count": 15, "avg_session_duration": 300, "categories_viewed": 5}
]
prediction = endpoint.predict(instances=instances)
print(f"Predicción CLV: ${prediction.predictions[0]:.2f}")

```

14.3 Pipelines Serverless

```

from google.cloud.aiplatform import PipelineJob

# Pipeline de ML completo
pipeline_definition = """
apiVersion: argoproj.io/v1alpha1
kind: Workflow
spec:

```

```

entrypoint: ml-pipeline
templates:
  - name: ml-pipeline
    dag:
      tasks:
        - name: data-preparation
          template: data-prep
        - name: model-training
          template: training
          dependsOn: data-preparation
        - name: model-evaluation
          template: evaluation
          dependsOn: model-training
        - name: model-deployment
          template: deployment
          dependsOn: model-evaluation
"""

# Ejecutar pipeline
pipeline = PipelineJob(
    display_name="techstore-ml-pipeline",
    template_path="pipeline.yaml",
    pipeline_root="gs://techstore-ml-staging/pipelines",
    parameter_values={
        "project": "techstore-analytics",
        "data_path": "gs://techstore-ml-staging/data"
    }
)

pipeline.run(sync=False)
print(f"Pipeline Job ID: {pipeline.resource_name}")

```

14.4 Ejercicios

1. **BigQuery ML:** Crea un modelo de regresión logística en BigQuery ML para predecir churn de clientes.
2. **Vertex AI AutoML:** Entrena un modelo de clasificación con AutoML y compáralo con un modelo custom de XGBoost.
3. **Endpoint:** Despliega un modelo en Vertex AI Endpoint y haz 10 predicciones online.
4. **Batch Prediction:** Configura una predicción batch para 100,000 clientes y mide el tiempo de ejecución.
5. **Pipeline:** Diseña un pipeline de Vertex AI Pipelines que incluya: data validation training evaluation deployment.

Capítulo 15: Proyecto Final: Plataforma de Datos con IA

15.1 Visión General

El proyecto final integra todo lo aprendido en la colección completa para construir una **Plataforma de Datos con IA** para TechStore. Esta plataforma combina:

1. **Ingesta de datos** (big data con PySpark)
2. **Almacenamiento** (data lake en capas)
3. **Modelos predictivos** (ML con scikit-learn/XGBoost)
4. **Forecasting** (series temporales con Prophet)
5. **NLP** (análisis de sentimiento de reseñas)
6. **IA Generativa** (RAG con LangChain)
7. **Despliegue** (Vertex AI + Streamlit)

15.2 Arquitectura

```
graph TD
  A[Datos Crudos] --> B[PySpark ETL]
  B --> C[Data Lake: Silver]
  C --> D[Feature Store]
  D --> E[ML Training]
  D --> F[Forecasting]
  D --> G[NLP Pipeline]
  E --> H[Model Registry]
  F --> H
  G --> H
  H --> I[Vertex AI Endpoint]
  I --> J[Streamlit Dashboard]
  J --> K[Usuarios de Negocio]
  J --> L[RAG Assistant]
```

15.3 Implementación

15.3.1 Capa de Datos

```

# 1. Ingesta con PySpark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, year, month, day

spark = SparkSession.builder.appName("TechStore-Platform").getOrCreate()

# Leer y transformar
bronze = spark.read.parquet("data/raw/techstore_transactions/")
silver = bronze \
    .filter(col("amount").isNotNull()) \
    .withColumn("year", year("timestamp")) \
    .withColumn("month", month("timestamp")) \
    .withColumn("day", day("timestamp"))

silver.write.mode("overwrite") \
    .partitionBy("year", "month") \
    .parquet("data/lake/silver/transactions/")

# 2. Feature Store
features = spark.sql("""
    SELECT
        customer_id,
        COUNT(*) as freq,
        SUM(amount) as total_spent,
        AVG(amount) as avg_ticket,
        MAX(amount) as max_ticket,
        DATEDIFF(MAX(timestamp), MIN(timestamp)) as recency_days
    FROM silver
    GROUP BY customer_id
""")
features.write.mode("overwrite").parquet("data/feature_store/customer_features/")

```

15.3.2 Modelos en Producción

```

# 3. Pipeline de ML
import joblib
import pandas as pd
from sklearn.ensemble import VotingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

df_features = pd.read_parquet("data/feature_store/customer_features/")
y = df_features["total_spent"]
X = df_features.drop(["customer_id", "total_spent"], axis=1)

# Ensemble final
modelo_final = VotingRegressor([
    ("xgb", XGBRegressor(n_estimators=200, learning_rate=0.05)),
    ("lgbm", LGBMRegressor(n_estimators=200, learning_rate=0.05)),
    ("rf", RandomForestRegressor(n_estimators=100))
])
modelo_final.fit(X, y)

```

```

joblib.dump(modelo_final, "models/techstore_ensemble.pkl")

# 4. Forecast Prophet
from prophet import Prophet
df_daily = spark.sql("""
    SELECT date_trunc('day', timestamp) as ds, SUM(amount) as y
    FROM silver GROUP BY ds
""").toPandas()

model_prophet = Prophet(yearly_seasonality=False, weekly_seasonality=True)
model_prophet.fit(df_daily)
future = model_prophet.make_future_dataframe(periods=30)
forecast = model_prophet.predict(future)

# 5. NLP Sentiment
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

reviews = pd.read_parquet("data/raw/techstore_reviews/")
vectorizer = TfidfVectorizer(max_features=2000)
X_vec = vectorizer.fit_transform(reviews["review_text"])
sentiment_model = LogisticRegression()
sentiment_model.fit(X_vec, (reviews["rating"] >= 4).astype(int))

joblib.dump(vectorizer, "models/vectorizer.pkl")
joblib.dump(sentiment_model, "models/sentiment.pkl")

```

15.3.3 RAG Assistant

```

# 6. Asistente RAG
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.load_local("data/vectorstore/techstore_docs", embeddings)

rag_assistant = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4", temperature=0.2),
    chain_type="stuff",
    retriever=vectorstore.as_retriever(k=5),
    return_source_documents=True
)

```

15.3.4 Dashboard Streamlit

```

# 7. Dashboard unificado
import streamlit as st
import pandas as pd
import plotly.express as px

```

```

import plotly.graph_objects as go
import joblib
from datetime import datetime, timedelta

st.set_page_config(page_title="TechStore AI Platform", layout="wide")
st.title("TechStore AI Platform")
st.markdown("Plataforma integrada de datos, ML e IA")

# Sidebar
pagina = st.sidebar.selectbox(
    "Módulo",
    ["Dashboard", "Predict CLV", "Forecast", "Sentiment", "RAG Assistant"]
)

# Cargar modelos
modelo_clv = joblib.load("models/techstore_ensemble.pkl")
model_prophet = joblib.load("models/prophet.pkl")
sentiment_model = joblib.load("models/sentiment.pkl")
vectorizer = joblib.load("models/vectorizer.pkl")

if pagina == "Dashboard":
    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Ventas Totales", "$12.5M", "+15%")
    col2.metric("Clientes Activos", "45,230", "+8%")
    col3.metric("Modelos en Prod", "4", "0")
    col4.metric("Forecast Accuracy", "92.3%", "+2.1%")

    # Gráfico de ventas
    df = pd.read_parquet("data/gold/ventas_diarias/")
    fig = px.line(df, x="fecha", y="ingresos", title="Ventas Diarias")
    st.plotly_chart(fig, use_container_width=True)

elif pagina == "Predict CLV":
    st.subheader("Predicción de CLV")
    col1, col2 = st.columns(2)
    with col1:
        freq = st.number_input("Frecuencia de compra", 1, 100, 15)
        avg_ticket = st.number_input("Ticket promedio ($)", 10, 5000, 150)
    with col2:
        recency = st.number_input("Días desde última compra", 0, 365, 7)
        max_ticket = st.number_input("Ticket máximo ($)", 10, 10000, 500)

    if st.button("Predecir CLV"):
        input_df = pd.DataFrame([freq, avg_ticket, recency, max_ticket]),
                                columns=["freq", "avg_ticket", "recency_days", "max_ticket"])
        pred = modelo_clv.predict(input_df)[0]
        st.success(f"CLV Predicho: **${pred:,.2f}**")

elif pagina == "RAG Assistant":
    st.subheader("Asistente TechStore")
    pregunta = st.text_input("Haz una pregunta sobre TechStore:")
    if pregunta:

```

```

with st.spinner("Analizando..."):
    respuesta = rag_assistant.invoke({"query": pregunta})
    st.write(respuesta["result"])
    with st.expander("Ver fuentes"):
        for doc in respuesta["source_documents"][:3]:
            st.info(doc.page_content[:200])

```

15.4 Despliegue

```

# Docker Compose para la plataforma
docker-compose.yml:
version: '3.8'
services:
    streamlit:
        build: ./app
        ports:
            - "8501:8501"
        volumes:
            - ./models:/app/models
            - ./data:/app/data

    mlflow:
        image: ghcr.io/mlflow/mlflow:v2.10.0
        ports:
            - "5000:5000"
        command: mlflow server --host 0.0.0.0

    vectorstore:
        image: chromadb/chroma:latest
        ports:
            - "8000:8000"

```

15.5 Ejercicios

1. **Complete el proyecto:** Implementa todos los componentes de la plataforma TechStore AI.
2. **Dashboard:** Añade 3 visualizaciones más al dashboard de Streamlit (mapa de calor, distribución de CLV, forecast plot).
3. **RAG Avanzado:** Añade filtros por categoría de producto y fecha al asistente RAG.
4. **Monitoreo:** Implementa logging de todas las predicciones y alertas de drift.
5. **Documentación:** Crea documentación técnica de la plataforma con diagrama de arquitectura y guía de uso.

Checkpoint 4: Plataforma de IA Integral

Objetivo

Construir y desplegar la plataforma completa que integra todos los componentes del Proyecto Final. Este checkpoint valida que la plataforma funciona end-to-end.

Arquitectura Completa

```
graph TB
    subgraph "Data Layer"
        A1[(TechStore Raw)]
        A2[(PySpark ETL)]
        A3[(Data Lake Silver)]
    end
    subgraph "ML Layer"
        B1[(Feature Store)]
        B2[(ML Training)]
        B3[(Model Registry)]
    end
    subgraph "AI Layer"
        C1[(RAG System)]
        C2[(LangChain Agent)]
        C3[(Vector Store)]
    end
    subgraph "App Layer"
        D1[(Streamlit App)]
        D2[(FastAPI Backend)]
        D3[(PostgreSQL)]
    end

    A1 --> A2 --> A3 --> B1
    B1 --> B2 --> B3
    B3 --> D2
    A3 --> C1 --> C2 --> D2
    C3 --> C1
    D2 --> D1
    D2 --> D3
```

Implementación

Fase 1: Verificar Componentes

```
import os
import joblib
import pandas as pd
from pathlib import Path

def verificar_plataforma():
    """Verificar que todos los componentes existen"""
    estado = {}

    # Data Lake
    estado["Data Lake"] = os.path.exists("data/lake/silver/transactions/")

    # Modelos
    estado["Modelo CLV"] = os.path.exists("models/techstore_ensemble.pkl")
    estado["Modelo Prophet"] = os.path.exists("models/prophet.pkl")
    estado["Modelo Sentiment"] = os.path.exists("models/sentiment.pkl")

    # Vector Store
    estado["Vector Store"] = os.path.exists("data/vectorstore/techstore_docs/")

    # Dashboard
    estado["Dashboard"] = os.path.exists("app/streamlit_app.py")

    return pd.Series(estado, name="Estado")

print("=== Verificación de Componentes ===")
for componente, existe in verificar_plataforma().items():
    status = "" if existe else ""
    print(f" {status} {componente}")
```

Fase 2: Validar Pipeline End-to-End

```
# Pipeline completo: datos → predicción → dashboard
def pipeline_completo(customer_id=42, producto="laptop", pregunta="¿Cómo mejorar las ventas?"):
    resultados = {}

    # 1. CLV Prediction
    modelo = joblib.load("models/techstore_ensemble.pkl")
    df = pd.read_parquet("data/feature_store/customer_features/")
    customer_data = df[df["customer_id"] == customer_id]
    if not customer_data.empty:
        pred = modelo.predict(customer_data.drop(["customer_id", "total_spent"], axis=1))
        resultados["clv_predicted"] = float(pred[0])
        resultados["clv_actual"] = float(customer_data["total_spent"].iloc[0])

    # 2. Sentiment Analysis
    sentiment = joblib.load("models/sentiment.pkl")
    vectorizer = joblib.load("models/vectorizer.pkl")
    reviews = pd.read_parquet("data/raw/techstore_reviews/")
    producto_reviews = reviews[reviews["review_text"].str.contains(producto, case=False, na=False)]
```

```

if not producto_reviews.empty:
    X = vectorizer.transform(producto_reviews["review_text"])
    predictions = sentiment.predict(X)
    resultados["sentiment_avg"] = float(predictions.mean())
    resultados["total_reviews"] = len(producto_reviews)

# 3. Forecast
forecast = pd.read_csv("data/gold/forecast_14d.csv")
resultados["forecast_14d_sum"] = float(forecast["yhat"].sum())

# 4. RAG Query
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.load_local("data/vectorstore/techstore_docs", embeddings)
qa = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4", temperature=0.2),
    chain_type="stuff",
    retriever=vectorstore.as_retriever(k=3)
)
resultados["rag_response"] = qa.invoke({"query": pregunta})["result"][:200]

return resultados

# Ejecutar
print("=== Pipeline End-to-End ===")
resultados = pipeline_completo()
for key, value in resultados.items():
    print(f" {key}: {value}")

```

Fase 3: API REST con FastAPI

```

# app/api.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib
import pandas as pd
import uvicorn

app = FastAPI(title="TechStore AI Platform API")

# Cargar modelos
modelo = joblib.load("models/techstore_ensemble.pkl")
sentiment = joblib.load("models/sentiment.pkl")
vectorizer = joblib.load("models/vectorizer.pkl")

class CLVRequest(BaseModel):
    freq: int
    avg_ticket: float

```

```

    recency_days: int
    max_ticket: float

class SentimentRequest(BaseModel):
    text: str

@app.get("/health")
def health():
    return {"status": "ok", "models": ["clv", "sentiment"]}

@app.post("/predict/clv")
def predict_clv(req: CLVRequest):
    input_df = pd.DataFrame([[
        req.freq, req.avg_ticket, req.recency_days, req.max_ticket
    ]], columns=["freq", "avg_ticket", "recency_days", "max_ticket"])
    pred = modelo.predict(input_df)[0]
    return {"predicted_clv": float(pred)}

@app.post("/predict/sentiment")
def predict_sentiment(req: SentimentRequest):
    X = vectorizer.transform([req.text])
    pred = sentiment.predict(X)[0]
    proba = sentiment.predict_proba(X)[0].tolist()
    return {
        "sentiment": "positive" if pred == 1 else "negative",
        "confidence": max(proba)
    }

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Fase 4: Dockerización

```

# Dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
EXPOSE 8501

CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]

```

```

# docker-compose.yml
version: '3.8'

services:

```

```

api:
  build: .
  ports:
    - "8000:8000"
  volumes:
    - ./models:/app/models
    - ./data:/app/data
  environment:
    - OPENAI_API_KEY=${OPENAI_API_KEY}

streamlit:
  build: .
  ports:
    - "8501:8501"
  command: streamlit run streamlit_app.py --server.port 8501
  volumes:
    - ./models:/app/models
    - ./data:/app/data
  depends_on:
    - api

mlflow:
  image: ghcr.io/mlflow/mlflow:v2.10.0
  ports:
    - "5000:5000"
  command: mlflow server --host 0.0.0.0 --backend-store-uri sqlite:///mlflow.db
  volumes:
    - mlflow_data:/mlflow

volumes:
  mlflow_data:

```

Validación Final

```

# Probar API
curl -X POST http://localhost:8000/predict/clv \
  -H "Content-Type: application/json" \
  -d '{"freq": 15, "avg_ticket": 150, "recency_days": 7, "max_ticket": 500}'

# Probar health
curl http://localhost:8000/health

```

Entregables

1. Código completo de la plataforma (ETL, ML, RAG, API, Dashboard)
2. Docker Compose funcional
3. API REST documentada (FastAPI + Swagger)
4. Dashboard Streamlit con 4+ módulos
5. Tests de integración end-to-end

Criterios de Evaluación

Criterio	Puntos
Todos los componentes funcionales	25
API REST con 3+ endpoints	20
Dashboard Streamlit completo	20
Dockerización funcional	15
Pipeline end-to-end probado	10
Documentación y tests	10

¡Has completado la plataforma de datos con IA más completa para TechStore!

Conclusión

Has llegado al final de *Big Data, ML e IA*, y con él, al final de la colección *Data & Analytics Mastery*.

El Viaje Completo

Cinco libros. Cientos de ejemplos de código. Miles de líneas de Python, SQL, DAX, YAML y Docker. Un dataset — TechStore — que ha evolucionado desde un simple CSV hasta una plataforma completa con IA generativa.

Libro 1: Los fundamentos — Python, SQL, estadística, pensamiento analítico.

Libro 2: La nube — BigQuery, pandas a escala, tu primer data warehouse.

Libro 3: Business Intelligence — Power BI, modelado dimensional, DAX, dashboards.

Libro 4: Ingeniería de Datos — dbt, Airflow, Docker, CI/CD, data lakes.

Libro 5: Big Data, ML e IA — PySpark, scikit-learn, TensorFlow, MLOps, LLMs, RAG.

Has pasado de analista a científico de datos, de ingeniero a arquitecto de IA.

Lo que Has Aprendido en Este Libro

Big Data

- Procesamiento distribuido con PySpark
- DataFrames, RDDs, Spark SQL y optimización
- Data lakes en capas (Bronze, Silver, Gold)
- Streaming con Structured Streaming y Kafka

Machine Learning

- Pipelines completos con scikit-learn
- Regresión, clasificación, clustering y recomendación
- XGBoost, Random Forest, Gradient Boosting
- Evaluación, validación cruzada y hyperparameter tuning

Deep Learning y NLP

- Redes neuronales con TensorFlow/Keras
- Procesamiento de lenguaje natural
- Word embeddings y LSTM para sentimiento

- Series temporales con ARIMA, Prophet y LSTM

MLOps e IA Generativa

- Experiment tracking con MLflow
- Data drift y monitoreo de modelos
- LLMs, RAG y LangChain
- Despliegue en Vertex AI y BigQuery ML

Producción

- APIs con FastAPI
- Dashboards con Streamlit
- Contenedores con Docker
- Plataforma completa integrada

El Futuro

La tecnología no se detiene. Aquí hay áreas para seguir explorando:

MLOps Avanzado: Kubeflow, TFX, feature stores como Feast.

IA en el Edge: Modelos ligeros para dispositivos móviles y IoT.

Data Mesh: Arquitectura distribuida de datos gobernada por dominios.

Agentes Autónomos: Multi-agent systems con LangGraph y AutoGen.

AI Responsible: Fairness, explicabilidad (SHAP, LIME), privacidad diferencial.

Una Reflexión Final

La tecnología cambia. Las herramientas se actualizan. Los frameworks aparecen y desaparecen. Pero los principios fundamentales — entender los datos, formular preguntas correctas, construir sistemas robustos, comunicar resultados — esos son eternos.

Has construido una base sólida en cada una de las áreas clave del ecosistema de datos moderno. Lo que viene después depende de ti: especialízate en una de estas áreas, profundiza, construye proyectos personales, contribuye a la comunidad.

El dataset TechStore te ha acompañado durante cinco libros. Pero el dataset más importante es el tuyo propio. Los datos de tu empresa, tu industria, tu pasión. Aplica lo aprendido, experimenta, falla rápido, itera.

Como dijo George Box: *"Todos los modelos son incorrectos, pero algunos son útiles."*

Tu trabajo como profesional de datos no es encontrar la verdad absoluta. Es encontrar modelos, sistemas e insights que sean **útiles**. Que ayuden a tomar mejores decisiones. Que automaticen procesos. Que creen valor.

Eso es lo que has aprendido a hacer en esta colección.

Gracias por acompañarme en este viaje de cinco libros. Ahora, sal y construye algo increíble con datos.

— Alex Goyzueta Delgado

Apéndice A: Soluciones a los Ejercicios

Capítulo 1

Ejercicio 1: Las 5 V del Big Data

Volumen: TechStore genera 10 millones de transacciones y 50 millones de eventos de navegación, demasiados para una sola máquina.

Velocidad: Los eventos de navegación llegan en tiempo real, requiriendo procesamiento streaming.

Variedad: Los datos incluyen transacciones estructuradas (CSV/Parquet), reseñas de texto no estructurado, eventos JSON semi-estructurados.

Veracidad: Reseñas con spam, datos incompletos, transacciones duplicadas requieren limpieza.

Valor: Predecir CLV, segmentar clientes, recomendar productos, analizar sentimiento.

Ejercicio 4: Generar eventos de navegación

```
eventos = spark.range(1_000_000) \
    .withColumn("user_id", (rand() * 100000).cast("int")) \
    .withColumn("page", when(rand() < 0.3, "home")
        .when(rand() < 0.6, "product")
        .when(rand() < 0.8, "cart")
        .otherwise("checkout")) \
    .withColumn("timestamp", expr("current_timestamp() - interval rand() * 30 days")) \
    .withColumn("duration_ms", (rand() * 30000 + 1000).cast("int"))
```

Capítulo 5

Ejercicio 1: Pipeline completo

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression

modelos = [
    ("Linear Regression", LinearRegression()),
    ("Random Forest", RandomForestRegressor(n_estimators=100)),
    ("Gradient Boosting", GradientBoostingRegressor(n_estimators=100))
```

```

]

for nombre, modelo in modelos:
    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("model", modelo)
    ])
    scores = cross_val_score(pipeline, X_train, y_train, cv=5, scoring="r2")
    print(f"{nombre}: R2 CV = {scores.mean():.3f} ± {scores.std():.3f}")

```

Capítulo 9

Ejercicio 2: Regularización

La mejor combinación suele ser **L2 + Dropout**. L2 (weight decay) penaliza pesos grandes, Dropout apaga neuronas aleatoriamente durante training. Juntos previenen overfitting mejor que cada uno por separado.

```

modelo_optimo = keras.Sequential([
    layers.Dense(128, activation="relu", kernel_regularizer=regularizers.l2(0.001)),
    layers.Dropout(0.3),
    layers.Dense(64, activation="relu", kernel_regularizer=regularizers.l2(0.001)),
    layers.Dropout(0.2),
    layers.Dense(1)
])

```

Capítulo 13

Ejercicio 2: Sistema RAG completo

```

from langchain.document_loaders import DirectoryLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

# Cargar documentación
loader = DirectoryLoader("docs/techstore/", glob="**/*.md", loader_cls=TextLoader)
docs = loader.load()

# Chunking
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

# Embeddings y vector store
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.from_documents(chunks, embeddings)

# QA Chain
qa = RetrievalQA.from_chain_type(

```

```

    llm=ChatOpenAI(model="gpt-4", temperature=0),
    chain_type="stuff",
    retriever=vectorstore.as_retriever(k=5),
    return_source_documents=True
)

# Preguntas de prueba
preguntas = [
    "¿Qué es el CLV y cómo se calcula?",
    "¿Cuáles son las categorías de productos?",
    "¿Cómo se segmentan los clientes?",
]
for p in preguntas:
    print(f"Q: {p}")
    print(f"A: {qa.invoke({'query': p})['result']}\n")

```

Apéndice B: Cheatsheets y Referencias Rápidas

PySpark

```
# SparkSession
spark = SparkSession.builder.appName("App").getOrCreate()

# Leer/Escribir
spark.read.parquet("path/")          # Parquet
spark.read.csv("path/", header=True) # CSV con cabecera
df.write.mode("overwrite").parquet("path/")
df.write.partitionBy("col").parquet("path/")

# Transformaciones
df.select("col1", "col2")
df.filter(col("col") > 100)
df.withColumn("new_col", expr("col * 2"))
df.groupBy("col").agg(sum("val"), avg("val"), count("*"))
df.join(other_df, "key", "left")
df.orderBy(col("val").desc())
df.dropDuplicates(["col"])
df.repartition(8, "key")

# SQL
df.createOrReplaceTempView("vista")
spark.sql("SELECT * FROM vista WHERE col > 100")

# Streaming
df_stream = spark.readStream.format("parquet").load("path/")
query = df_stream.writeStream \
    .format("console") \
    .trigger(processingTime="10 seconds") \
    .start()
query.awaitTermination()
```

Scikit-learn

```
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# Pipeline
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(), cat_cols)
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("clf", RandomForestClassifier())
])

# Entrenar
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model.fit(X_train, y_train)

# Evaluar
score = model.score(X_test, y_test)
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]

# Cross-validation
scores = cross_val_score(model, X, y, cv=5, scoring="f1")

# Grid Search
GridSearchCV(model, param_grid, cv=3, scoring="roc_auc")

```

TensorFlow/Keras

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# MLP
model = keras.Sequential([
    layers.Dense(128, activation="relu", input_shape=(n_features,)),
    layers.Dropout(0.3),
    layers.Dense(64, activation="relu"),
    layers.Dense(1) # regresión
    # layers.Dense(1, activation="sigmoid") # clasificación binaria
])

model.compile(optimizer="adam", loss="mse", metrics=["mae"])
model.fit(X_train, y_train, epochs=100, validation_split=0.2,
        callbacks=[EarlyStopping(patience=5)])
model.evaluate(X_test, y_test)
model.predict(X_test)

# LSTM

```

```
layers.LSTM(64, return_sequences=True)
layers.Bidirectional(LSTM(64))
```

MLflow

```
import mlflow

mlflow.set_experiment("experimento")

with mlflow.start_run():
    mlflow.log_param("n_estimators", 100)
    mlflow.log_metric("accuracy", 0.95)
    mlflow.log_artifact("plot.png")
    mlflow.sklearn.log_model(model, "model")

# Cargar modelo
model = mlflow.sklearn.load_model("runs:/run_id/model")

# Model Registry
mlflow.register_model("runs:/run_id/model", "ModelName")
```

LangChain

```
from langchain.chains import RetrievalQA
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_openai import ChatOpenAI

# RAG
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.from_documents(docs, embeddings)
qa = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4"),
    chain_type="stuff",
    retriever=vectorstore.as_retriever(k=5)
)
qa.invoke({"query": "pregunta"})
```

Docker

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "api:app", "--host", "0.0.0.0"]
```

```
docker build -t techstore-ai .
docker run -p 8000:8000 techstore-ai
```

```
docker-compose up -d  
docker-compose down
```

Apéndice C: Guía de Instalación y Configuración

C.1 Python y Entorno Virtual

```
# Python 3.10+ requerido
python --version

# Crear entorno virtual
python -m venv .venv

# Activar
# Windows:
.venv\Scripts\activate
# Linux/Mac:
source .venv/bin/activate

# Actualizar pip
pip install --upgrade pip
```

C.2 Instalación de Librerías

C.2.1 Data Science Base

```
pip install pandas numpy matplotlib seaborn scikit-learn
```

C.2.2 Big Data

```
pip install pyspark
# Opcional: conectar con Hadoop
# pip installhadoop
```

C.2.3 Machine Learning

```
pip install xgboost lightgbm
pip install scikit-learn # ya incluido arriba
```

C.2.4 Deep Learning

```
pip install tensorflow
```

```
# Para GPU:  
# pip install tensorflow[and-cuda]
```

C.2.5 NLP

```
pip install nltk wordcloud gensim  
  
# Descargar recursos NLTK  
python -c "import nltk; nltk.download('punkt'); nltk.download('stopwords'); nltk.download('wordnet')"
```

C.2.6 Series Temporales

```
pip install prophet statsmodels
```

C.2.7 MLOps

```
pip install mlflow
```

C.2.8 IA Generativa

```
pip install langchain langchain-community langchain-openai  
pip install langchain-google-vertexai  
pip install chromadb faiss-cpu  
pip install huggingface-hub sentence-transformers
```

C.2.9 Producción y API

```
pip install fastapi uvicorn streamlit  
pip install google-cloud-aiplatform google-cloud-bigquery
```

C.2.10 Instalación Completa

```
pip install -r requirements.txt
```

Contenido de `requirements.txt`:

```
pandas>=2.0.0  
numpy>=1.24.0  
matplotlib>=3.7.0  
seaborn>=0.12.0  
scikit-learn>=1.3.0  
pyspark>=3.5.0  
xgboost>=2.0.0  
lightgbm>=4.0.0  
tensorflow>=2.15.0  
nltk>=3.8.0  
wordcloud>=1.9.0  
gensim>=4.3.0  
prophet>=1.1.0  
statsmodels>=0.14.0  
mlflow>=2.10.0  
langchain>=0.1.0
```

```
langchain-community>=0.0.10
langchain-openai>=0.0.2
chromadb>=0.4.0
faiss-cpu>=1.7.0
sentence-transformers>=2.2.0
fastapi>=0.109.0
uvicorn>=0.27.0
streamlit>=1.30.0
google-cloud-aiplatform>=1.38.0
google-cloud-bigquery>=3.14.0
```

C.3 Configuración de PySpark

```
import os

os.environ["PYSPARK_PYTHON"] = "python"
os.environ["PYSPARK_DRIVER_PYTHON"] = "python"

# Para Windows:
os.environ["HADOOP_HOME"] = "C:\\\\hadoop"
# Descargar winutils.exe de https://github.com/steveloughran/winutils
```

C.4 Configuración de APIs

C.4.1 OpenAI

```
export OPENAI_API_KEY="sk-..."
# 0 crear .env:
echo "OPENAI_API_KEY=sk-..." > .env
```

C.4.2 Google Cloud

```
# Instalar gcloud CLI
gcloud auth application-default login
gcloud config set project techstore-analytics

# 0 usar service account
export GOOGLE_APPLICATION_CREDENTIALS="service-account.json"
```

C.5 Docker

```
# Instalar Docker Desktop desde https://www.docker.com/products/docker-desktop/

# Verificar
docker --version
docker-compose --version

# Comandos básicos
docker pull apache/spark:latest
docker pull minio/minio
```

```
docker pull apache/airflow:latest
```

C.6 Verificación de Instalación

```
# scripts/verify_install.py
import sys
import importlib

librerias = [
    "pandas", "numpy", "sklearn", "pyspark", "xgboost",
    "tensorflow", "nltk", "prophet", "mlflow", "langchain",
    "fastapi", "uvicorn", "streamlit"
]

print(f"Python {sys.version}\n")
print("=== Verificación de Librerías ===")

for lib in librerias:
    try:
        importlib.import_module(lib)
        print(f"  {lib}")
    except ImportError:
        print(f"  {lib}")

print("\n¡Listo para comenzar!")
```

C.7 Troubleshooting

Problema	Solución
PySpark no encuentra Hadoop	Descargar winutils.exe o instalar en WSL
TensorFlow GPU no detecta	`pip install tensorflow[and-cuda]`
Prophet falla en Windows	Usar WSL o conda install
mlflow no conecta	Verificar `mlflow.set_tracking_uri()`
LangChain OpenAI sin API key	Configurar OPENAI_API_KEY en .env
Streamlit no muestra gráficos	`pip install --upgrade plotly`
FAISS no se instala	Usar `pip install faiss-cpu` (no faiss)

Sobre el Autor

Alex Goyzueta Delgado es un profesional apasionado por los datos, la tecnología y la educación. Con años de experiencia práctica en análisis de datos, business intelligence, data engineering y machine learning, ha trabajado ayudando a empresas a transformar datos en decisiones estratégicas.

Trayectoria

La carrera de Alex abarca todo el ecosistema de datos:

- **Análisis de datos:** Python, SQL, estadística, visualización
- **Business Intelligence:** Power BI, Tableau, modelado dimensional, DAX
- **Data Engineering:** dbt, Airflow, Docker, CI/CD, data lakes
- **Big Data:** PySpark, Spark SQL, streaming, optimización
- **Machine Learning:** scikit-learn, XGBoost, TensorFlow, MLOps
- **IA Generativa:** LLMs, RAG, LangChain, Vertex AI

La Colección Data & Analytics Mastery

Esta colección de 5 libros es el resultado de años de experiencia y la convicción de que el conocimiento en datos debe ser accesible, práctico y progresivo:

1. **Los Fundamentos del Analista de Datos** — La base sólida que todo profesional necesita
2. **Python para Datos y tu Primer Data Warehouse Cloud** — De la teoría a la nube
3. **Business Intelligence y Modelado de Datos** — Dashboards que transforman negocios
4. **Ingeniería de Datos Moderna** — Pipelines robustos y escalables
5. **Big Data, ML e IA** — El futuro de los datos, hoy

Filosofía

Alex cree en el aprendizaje práctico basado en proyectos. Cada concepto se introduce en contexto, se aplica inmediatamente y se refuerza con ejercicios. El dataset TechStore, creado específicamente para esta colección, proporciona un hilo conductor que permite ver la evolución de los datos a través de todas las etapas del ecosistema.

Contacto

- Email: alex.goyzueta@email.com

"Los datos no son solo números y tablas. Son historias esperando ser contadas, decisiones esperando ser tomadas, y oportunidades esperando ser descubiertas."